



БАЗЫ ДАННЫХ

Лекция 3.

Индексы

Методы доступа к данным

Виды индексов в PG

Поиск

КодСотр	Фамилия	Отдел	Адрес
73	Иванов	04	г.Минск ...
75	Степанов	04	г.Минск ...
80	Кустов	21	г.Минск ...
66	Чернов	22	г.Минск ...
06	Трусов	14	г.Минск ...
03	Терпухов	12	г.Минск ...
83	Павлов	04	г.Минск ...
90	Мухина	12	г.Минск ...
91	Артемьев	21	г.Минск ...
63	Васильев	22	г.Минск ...
84	Васильева	04	г.Минск ...
81	Уваров	04	г.Минск ...
98	Шишкин	21	г.Минск ...
92	Удалов	22	г.Минск ...

Найти
сотрудника
с фамилией
Павлов

Поиск

КодСотр	Фамилия	Отдел	Адрес
73	Иванов	04	г.Минск ...
75	Степанов	04	г.Минск ...
80	Кустов	21	г.Минск ...
66	Чернов	22	г.Минск ...
06	Трусов	14	г.Минск ...
03	Терпухов	12	г.Минск ...
83	Павлов	04	г.Минск ...
90	Мухина	12	г.Минск ...
91	Артемьев	21	г.Минск ...
63	Васильев	22	г.Минск ...
84	Васильева	04	г.Минск ...
81	Уваров	04	г.Минск ...
98	Шишкин	21	г.Минск ...
92	Удалов	22	г.Минск ...

Найти
сотрудника
с фамилией
Павлов

Перебор 7 строк

Поиск

КодСотр	Фамилия	Отдел	Адрес
73	Иванов	04	г.Минск ...
75	Степанов	04	г.Минск ...
80	Кустов	21	г.Минск ...
66	Чернов	22	г.Минск ...
06	Трусов	14	г.Минск ...
03	Терпухов	12	г.Минск ...
83	Павлов	04	г.Минск ...
90	Мухина	12	г.Минск ...
91	Артемьев	21	г.Минск ...
63	Васильев	22	г.Минск ...
84	Васильева	04	г.Минск ...
81	Уваров	04	г.Минск ...
98	Шишкин	21	г.Минск ...
92	Удалов	22	г.Минск ...

Найти
сотрудников
из отдела 22

Поиск

КодСотр	Фамилия	Отдел	Адрес
73	Иванов	04	г.Минск ...
75	Степанов	04	г.Минск ...
80	Кустов	21	г.Минск ...
66	Чернов	22	г.Минск ...
06	Трусов	14	г.Минск ...
03	Терпухов	12	г.Минск ...
83	Павлов	04	г.Минск ...
90	Мухина	12	г.Минск ...
91	Артемьев	21	г.Минск ...
63	Васильев	22	г.Минск ...
84	Васильева	04	г.Минск ...
81	Уваров	04	г.Минск ...
98	Шишкин	21	г.Минск ...
92	Удалов	22	г.Минск ...

Найти
сотрудников
из отдела 22

Перебор всех строк

Поиск

КодСотр	Фамилия	Отдел	Адрес
91	Артемов	21	г.Минск ...
63	Васильев	22	г.Минск ...
84	Васильева	04	г.Минск ...
73	Иванов	04	г.Минск ...
80	Кустов	21	г.Минск ...
90	Мухина	12	г.Минск ...
83	Павлов	04	г.Минск ...
75	Степанов	04	г.Минск ...
03	Терпухов	12	г.Минск ...
06	Трусов	14	г.Минск ...
81	Уваров	04	г.Минск ...
92	Удалов	22	г.Минск ...
66	Чернов	22	г.Минск ...
98	Шишкин	21	г.Минск ...

Найти
сотрудника
с фамилией
Удалов

Таблица
упорядочена
по полю Фамилия

Бинарный поиск

КодСотр	Фамилия	Отдел	Адрес
91	Артемов	21	г.Минск ...
63	Васильев	22	г.Минск ...
84	Васильева	04	г.Минск ...
73	Иванов	04	г.Минск ...
80	Кустов	21	г.Минск ...
90	Мухина	12	г.Минск ...
83	Павлов	04	г.Минск ...
75	Степанов	04	г.Минск ...
03	Терпухов	12	г.Минск ...
06	Трусов	14	г.Минск ...
81	Уваров	04	г.Минск ...
92	Удалов	22	г.Минск ...
66	Чернов	22	г.Минск ...
98	Шишкин	21	г.Минск ...



Найти
сотрудника
с фамилией
Удалов

Таблица
упорядочена
по полю Фамилия

Бинарный поиск



КодСотр	Фамилия	Отдел	Адрес
91	Артемов	21	г.Минск ...
63	Васильев	22	г.Минск ...
84	Васильева	04	г.Минск ...
73	Иванов	04	г.Минск ...
80	Кустов	21	г.Минск ...
90	Мухина	12	г.Минск ...
83	Павлов	04	г.Минск ...
75	Степанов	04	г.Минск ...
03	Терпухов	12	г.Минск ...
06	Трусов	14	г.Минск ...
81	Уваров	04	г.Минск ...
92	Удалов	22	г.Минск ...
66	Чернов	22	г.Минск ...
98	Шишкин	21	г.Минск ...

Найти
сотрудника
с фамилией
Удалов

Таблица
упорядочена
по полю Фамилия

Бинарный поиск

КодСотр	Фамилия	Отдел	Адрес
91	Артемов	21	г.Минск ...
63	Васильев	22	г.Минск ...
84	Васильева	04	г.Минск ...
73	Иванов	04	г.Минск ...
80	Кустов	21	г.Минск ...
90	Мухина	12	г.Минск ...
83	Павлов	04	г.Минск ...
75	Степанов	04	г.Минск ...
03	Терпухов	12	г.Минск ...
06	Трусов	14	г.Минск ...
81	Уваров	04	г.Минск ...
92	Удалов	22	г.Минск ...
66	Чернов	22	г.Минск ...
98	Шишкин	21	г.Минск ...

Найти
сотрудника
с фамилией
Удалов

Таблица
упорядочена
по полю Фамилия

Бинарный поиск



КодСотр	Фамилия	Отдел	Адрес
91	Артемов	21	г.Минск ...
63	Васильев	22	г.Минск ...
84	Васильева	04	г.Минск ...
73	Иванов	04	г.Минск ...
80	Кустов	21	г.Минск ...
90	Мухина	12	г.Минск ...
83	Павлов	04	г.Минск ...
75	Степанов	04	г.Минск ...
03	Терпухов	12	г.Минск ...
06	Трусов	14	г.Минск ...
81	Уваров	04	г.Минск ...
92	Удалов	22	г.Минск ...
66	Чернов	22	г.Минск ...
98	Шишкин	21	г.Минск ...

Найти
сотрудника
с фамилией
Удалов

Таблица
упорядочена
по полю Фамилия

Понятие индекса

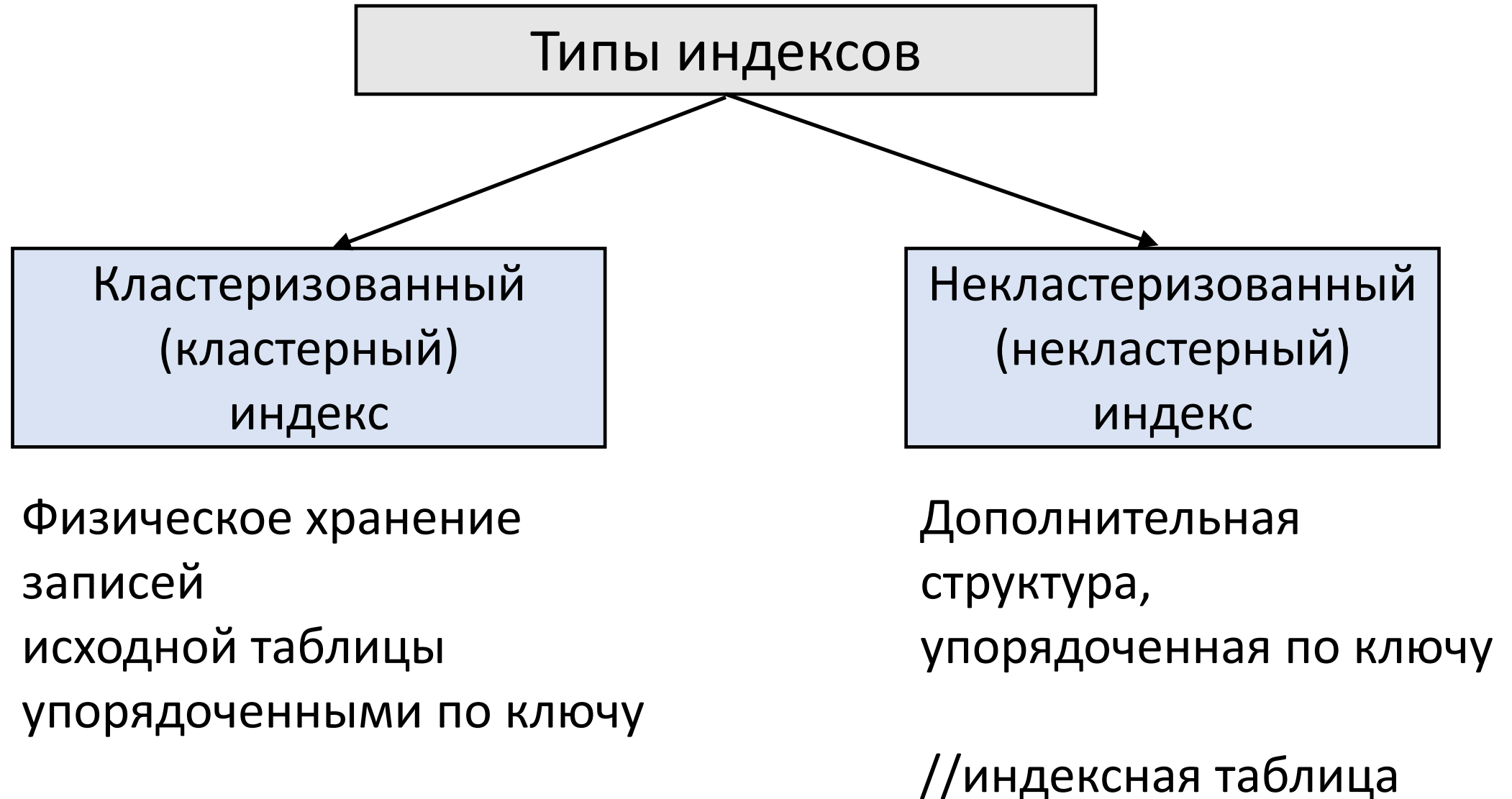
- Основная проблема в СУБД – поиск нужных данных за минимальное время.
- Быстрый поиск может быть выполнен в случае, если данные отсортированы.
- Постоянная сортировка данных в таблице невозможна + критериев поиска может быть несколько.

Индекс – это структура данных для быстрого поиска записей в таблице по значению ключа. Индексы могут создаваться для каждого критерия поиска.

Для чего нужна индексация

- для быстрого поиска строк по условию WHERE,
- для уменьшения числа отбираемых строк (выбирается индекс, по которому отсеивается максимальное число строк),
- для получения строк из других таблиц во время объединений (JOIN),
- для поиска минимального/максимального (MIN() или MAX()) значения в столбце (эти константы сохраняются при индексировании),
- для быстрой сортировки/группировки таблицы по проиндексированным столбцам.

Методы организации индекса



Кластеризованный индекс

Кластеризованный индекс — структура данных, при которой значения индекса хранятся вместе с данными, им соответствующими.

- И индексы, и данные упорядочены.
- При добавлении новой строки в таблицу, она дописывается в нужную ветку структуры, соответствующую ей по сортировке.
- Всегда поддерживается упорядоченность (по ключу)
- Может потребоваться перестроение (может и нет!)

Кластеризованный индекс

Таблица индекса

Содержит индексные записи в отсортированном порядке, расположенные в блоках.

Индексная запись

Индексный ключ

Номер блока в основном файле

Основная таблица

Содержит последовательность записей одинаковой длины в отсортированном порядке по индексному ключу, расположенные в блоках, имеющих изначально свободное пространство.

Поиск данных осуществляется по индексному ключу в индексном файле методом бинарного поиска по блокам.

Кластеризованный индекс

Таблица индекса

КодСотр	Номер блока
03	1
73	2
83	3
98	4

Блок 1

Блок 2

Блок 1

Блок 2

Блок 3

Основная таблица

КодСотр	Фамилия	отдел	Адрес
03	Терпухов	12	г.Минск ...
06	Трусов	14	г.Минск ...
63	Васильев	22	г.Минск ...
66	Чернов	22	г.Минск ...
73	Иванов	4	г.Минск ...
75	Степанов	4	г.Минск ...
80	Кустов	21	г.Минск ...
81	Уваров	4	г.Минск ...
83	Павлов	4	г.Минск ...
84	Васильева	4	г.Минск ...
90	Мухина	12	г.Минск ...
91	Артемьев	21	г.Минск ...
92	Удалов	22	г.Минск ...
98	Шишкин	21	г.Минск ...

Некластеризованный индекс

Некластеризованный индекс — отдельная структура данных в базе данных, которая ускоряет поиск по таблице, но не меняет физический порядок самих строк на диске.

- Индексы упорядочены, данные в таблице — нет
- При добавлении новой строки в таблицу, ее значение ключа дописывается в нужную ветку индексной структуры, соответствующую ей по сортировке.
- Не требуется перестроение, потому что данные лежат в свободном порядке

Некластеризованный индекс

Таблица индекса

Содержит индексные записи в отсортированном порядке, расположенные в блоках, имеющих изначально свободное пространство.

Основная таблица

Содержит последовательность записей одинаковой длины в произвольном порядке.

Индексная запись

Индексный ключ
Номер записи в основном файле

Поиск данных осуществляется по индексному ключу в индексной таблице методом бинарного поиска по блокам.

Некластеризованный индекс

Таблица индекса

Блок 1

Блок 2

Блок 3

Блок 4

КодСотр	Номер записи
03	6
06	5
63	10
66	4
73	1
75	2
80	3
81	12
83	7
84	11
90	8
91	9
92	14
98	13

Основная таблица

КодСотр	Фамилия	отдел	Адрес
73	Иванов	04	г.Минск ...
75	Степанов	04	г.Минск ...
80	Кустов	21	г.Минск ...
66	Чернов	22	г.Минск ...
06	Трусов	14	г.Минск ...
03	Терпухов	12	г.Минск ...
83	Павлов	04	г.Минск ...
90	Мухина	12	г.Минск ...
91	Артемьев	21	г.Минск ...
63	Васильев	22	г.Минск ...
84	Васильева	04	г.Минск ...
81	Уваров	04	г.Минск ...
98	Шишкин	21	г.Минск ...
92	Удалов	22	г.Минск ...

Некластеризованный индекс

2-я таблица индекса

Блок 1

Блок 2

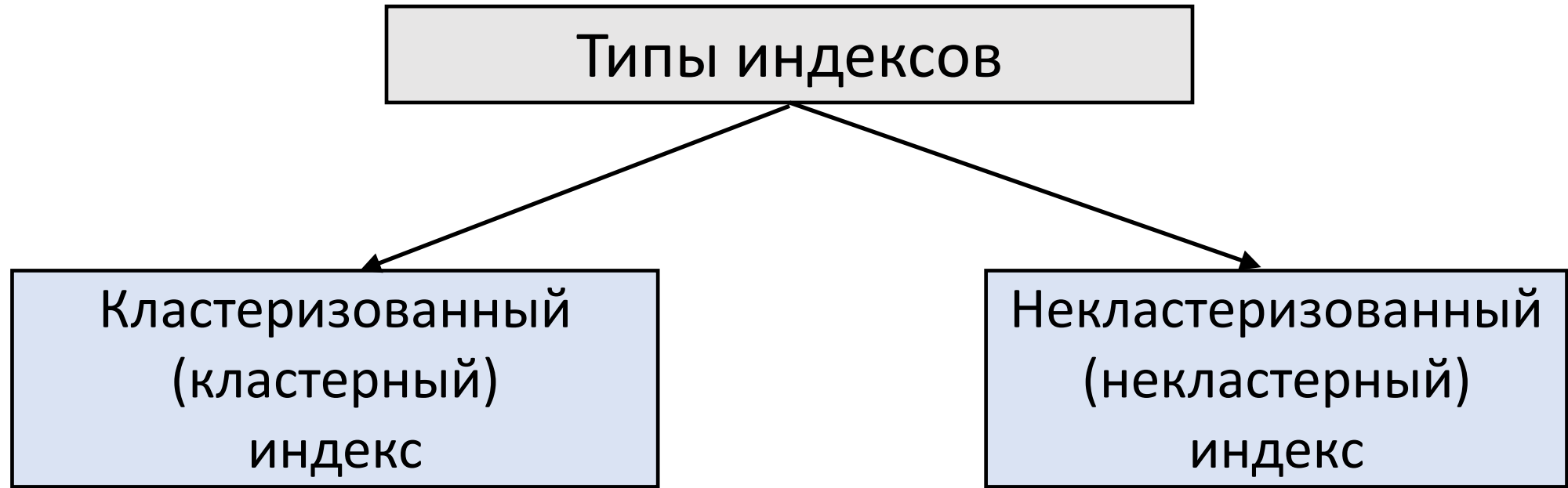
Блок 3

Фамилия	Номер записи
Артемьев	9
Васильев	10
Васильева	11
Иванов	1
Кустов	3
Мухина	8
Павлов	7
Степанов	2
Терпухов	6
Трусов	5
Уваров	12
Удалов	14
Чернов	4
Шишкин	13

Основная таблица

КодСотр	Фамилия	отдел	Адрес
73	Иванов	04	г.Минск ...
75	Степанов	04	г.Минск ...
80	Кустов	21	г.Минск ...
66	Чернов	22	г.Минск ...
06	Трусов	14	г.Минск ...
03	Терпухов	12	г.Минск ...
83	Павлов	04	г.Минск ...
90	Мухина	12	г.Минск ...
91	Артемьев	21	г.Минск ...
63	Васильев	22	г.Минск ...
84	Васильева	04	г.Минск ...
81	Уваров	04	г.Минск ...
98	Шишкин	21	г.Минск ...
92	Удалов	22	г.Минск ...

Методы организации индекса



По умолчанию для первичного ключа в некоторых СУБД (MySQL, Microsoft SQL Server)

В PostgreSQL нет постоянных автоматических кластерных индексов

По умолчанию в PostgreSQL – для первичного ключа и при создании индексов

Сравнение кластерного и некластерного индексов

Критерий	Кластеризованный	Некластеризованный
Порядок хранения	Сортирует и хранит данные по правилу <u>сортировки</u> ; использует первичный ключ для структуризации данных в таблице; всегда изменяет физический порядок хранимых данных	<u>Не упорядочивает</u> физическое хранение, используя метки для доступа к данным; создает свою отдельную структуру данных, которая не зависит от фактической таблицы, для которой он строится
Конечные листья индекса	Используются для хранения записей	Не используются для хранения записей
Количество индексов в таблице	Если есть, то ровно один	Не ограничено

Сравнение кластерного и некластерного индексов

Критерий	Кластеризованный	Некластеризованный
Место на диске	Занимает много места	Занимает мало места
Скорость доступа	Быстрый	Медленный
Доп. место на диске	Не нужно	Необходимо, так как индекс хранится отдельно от физических данных таблицы
Простота использования	Не требует явного объявления и создается по умолчанию при определении ключа, то есть в принципе легче в использовании — в том числе в плане синтаксиса KROME PostgreSQL	Требует явного определения ; содержит в себе только те столбцы таблицы, по которым определен, поэтому системе запросов нужна дополнительная операция для того, чтобы получить данные из индекса

Индекс первичного ключа в PostgreSQL

- В PostgreSQL индекс первичного ключа по умолчанию не является кластерным.
- Может стать таковым после выполнения команды CLUSTER, которая физически упорядочивает данные таблицы в соответствии с выбранным индексом.
- Команда CLUSTER перестраивает таблицу единожды и далее **не поддерживает** упорядоченность при добавлении новых записей.

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(100));
```

```
CLUSTER users USING users_pkey;
```



БАЗЫ ДАННЫХ

Лекция 3.

Методы доступа к данным

Методы доступа к данным

Последовательное сканирование (Seq Scan)

Сканирование индекса (Index Scan)

Сканирование по битовой карте (Bitmap Scan)

Сканирование только индекса (Index-Only Scan)

Сравнение эффективности методов доступа

Основано на материалах курса PostgresPro QPT-16

<https://postgrespro.ru/education/courses/QPT>



Методы доступа к данным

// EXPLAIN ANALYZE – то же самое,
но запрос реально выполняется

План запроса

```
=> EXPLAIN (buffers) SELECT * FROM flights;
```

QUERY PLAN

```
-----  
Seq Scan on flights (cost=0.00..4772.67 rows=214867 width=63)  
Planning:  
  Buffers: shared hit=84 read=14 dirtied=3  
(3 rows)
```

Методы доступа к данным

```
=> EXPLAIN (buffers) SELECT * FROM flights;
```

QUERY PLAN

```
-----  
Seq Scan on flights (cost=0.00..4772.67 rows=214867 width=63)  
Planning:  
  Buffers: shared hit=84 read=14 dirtied=3  
(3 rows)
```

- cost — оценка стоимости;
- rows — оценка числа строк, возвращаемых операцией;
- width — оценка среднего размера одной строки в байтах.

Стоимость:

Первое число — начальная стоимость вычисления узла.

Второе число — полную стоимость получения всех данных.

Seq Scan

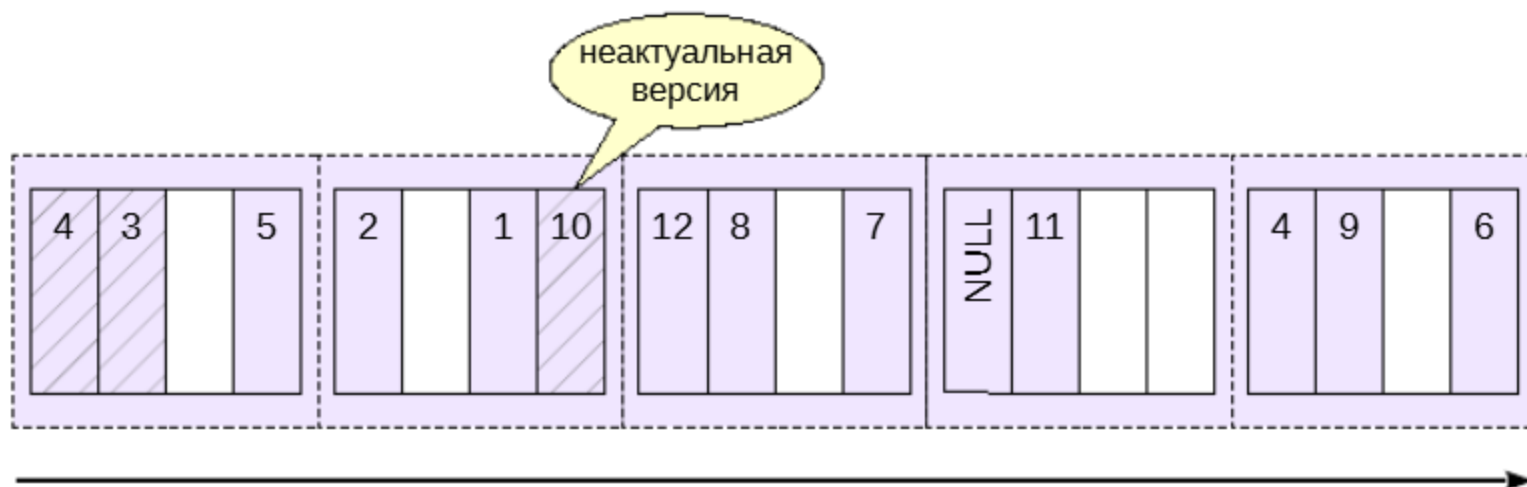
Последовательное чтение всех страниц

страницы читаются в кеш

проверяется видимость версий строк

данные возвращаются в произвольном порядке

время сканирования зависит от физического размера файла



Типы индексов в PG

- B-tree
- Хеш-индекс
- GiST
- Класс операторов
- SP-GiST
- GIN
- BRIN

Индекс типа B-tree
создается в PG по
умолчанию
и наиболее часто
применяется на практике

Индекс

вспомогательная структура во внешней памяти
сопоставляет ключи и идентификаторы строк таблицы

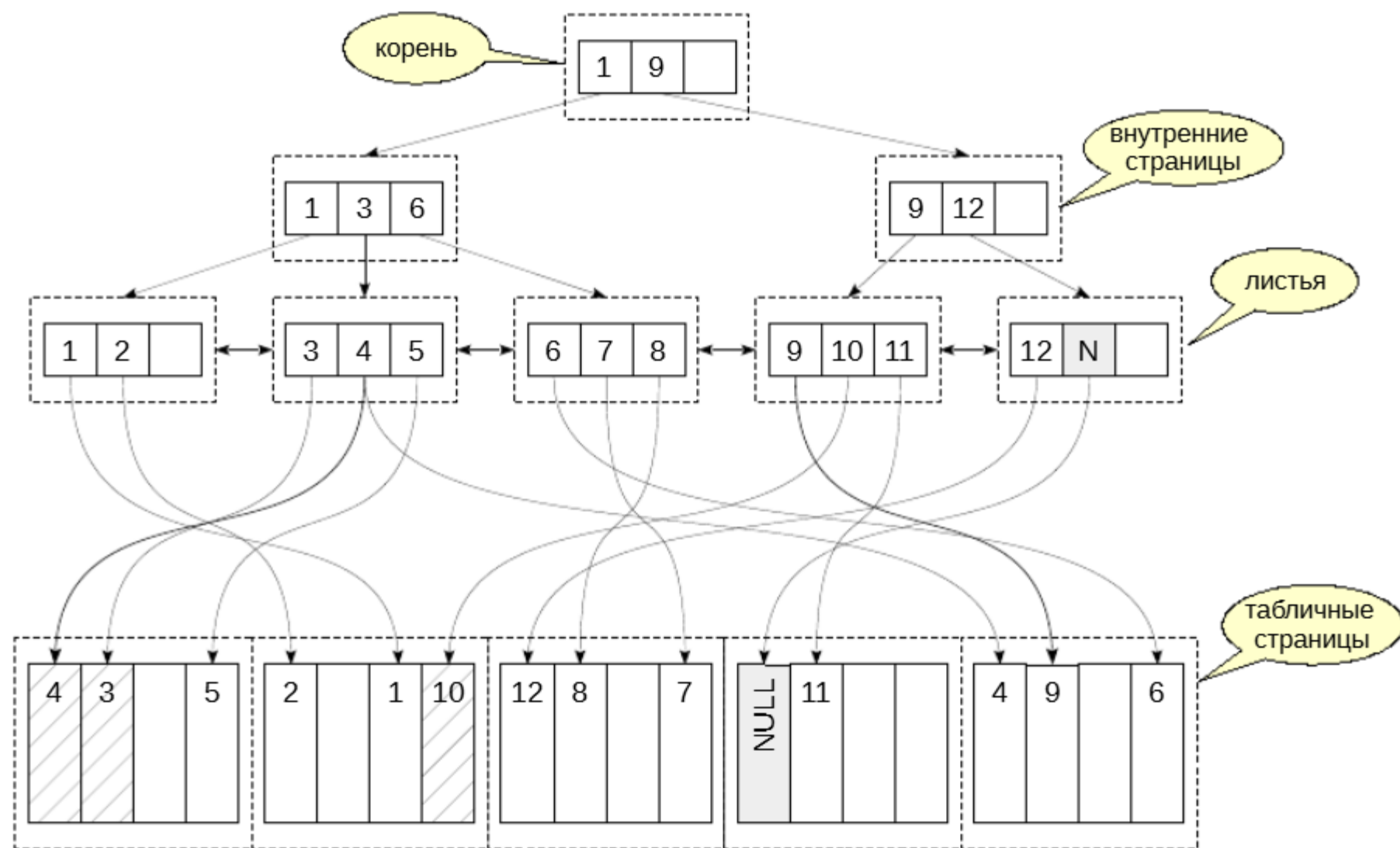
Устройство: дерево поиска

сбалансированное
сильно ветвистое
только сортируемые типы данных (операции «больше», «меньше»)
результаты поиска автоматически отсортированы

Использование

ускорение доступа
поддержка ограничений целостности

В-дерево

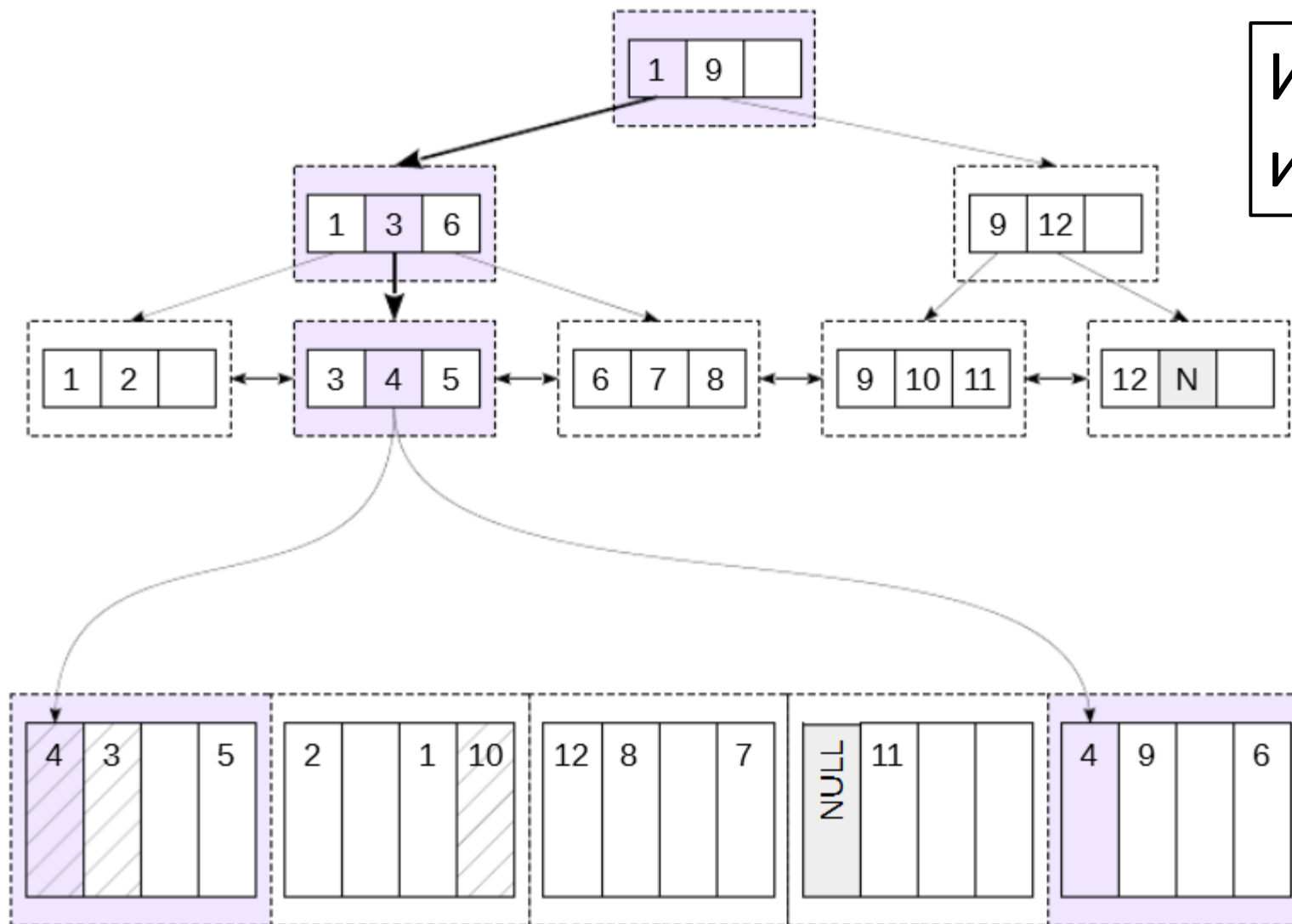


B-tree

- Индексная страница может быть заполнена частично. Свободное место используется для вставки в индекс новых записей. Если же на странице не хватает места — она разделяется на две новых страницы.
- Разделившиеся страницы никогда не объединяются, и это в некоторых случаях может приводить к разрастанию индекса.
- По умолчанию считается, что неопределенные значения «больше» обычных, и они хранятся в дереве справа. Этот порядок можно изменить при создании индекса с помощью предложений `NULLS LAST` и `NULLS FIRST`.

Index Scan: одно значение

Ищем значение
индекса 4



=> **EXPLAIN SELECT** * **FROM** bookings **WHERE** book_ref = 'CDE08B';

QUERY PLAN

Index Scan using bookings_pkey on bookings (cost=0.43..8.45 rows=1 width=16)
Index Cond: (book_ref = 'CDE08B'::bpchar)
(2 rows)

=> EXPLAIN

SELECT * FROM bookings

WHERE book_ref = 'CDE08B' AND total_amount > 1000;

QUERY PLAN

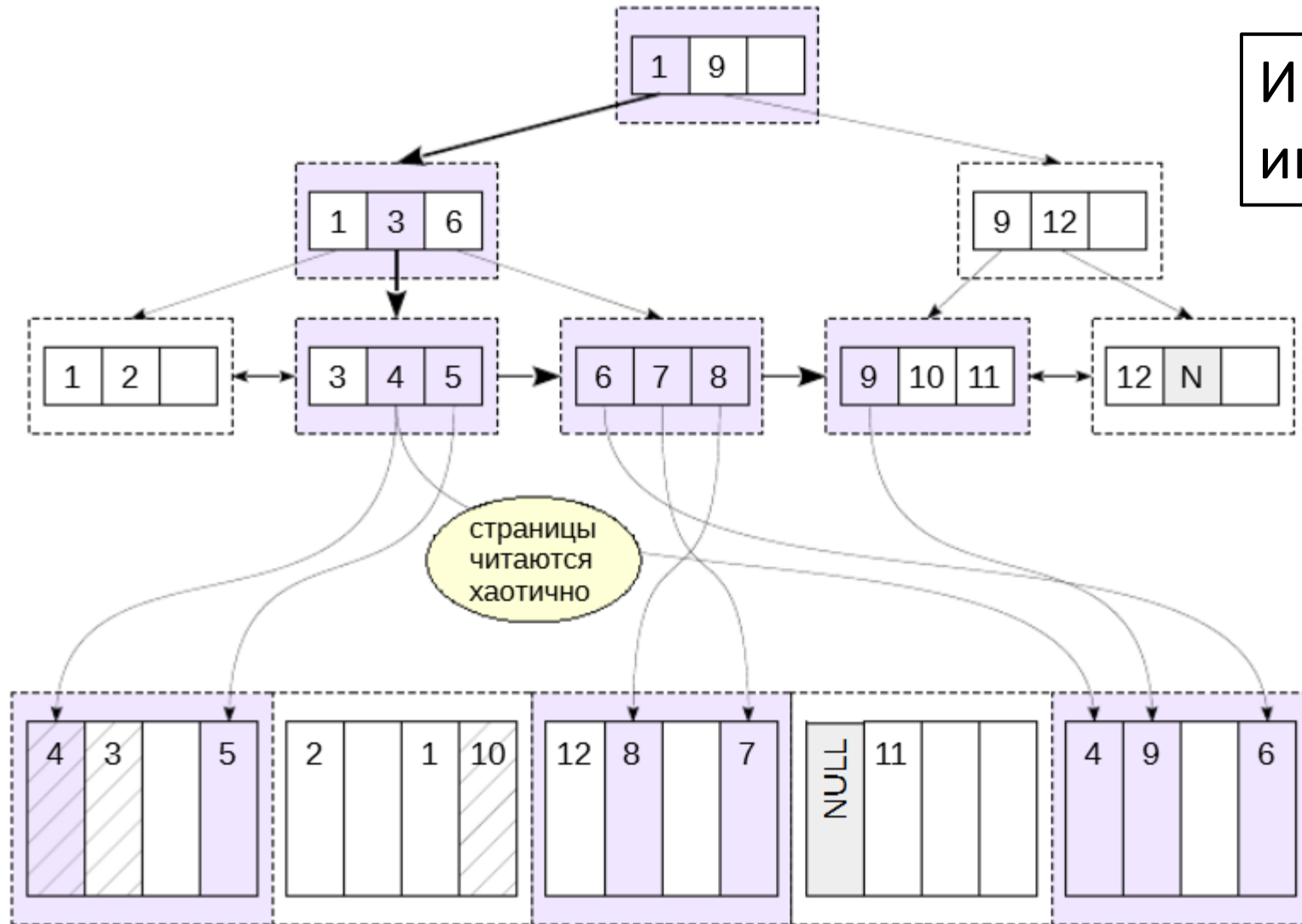
Index Scan using bookings_pkey on bookings (cost=0.43..8.45 rows=1 width=16)

Index Cond: (book_ref = 'CDE08B'::bpchar)

Filter: (total_amount > '1000'::numeric)

(3 rows)

Index Scan: диапазон



```
=> EXPLAIN (costs off)
SELECT * FROM bookings
WHERE book_ref > '000900' AND book_ref < '000939'
ORDER BY book_ref;
```

Диапазон по индексу

QUERY PLAN

Index Scan using bookings_pkey on bookings
Index Cond: ((book_ref > '000900'::bpchar) AND (book_ref < '000939'::b
(2 rows)

```
=> EXPLAIN (costs off)
SELECT * FROM bookings
WHERE book_ref > '000900' AND book_ref < '000939'
ORDER BY book_ref;
```

QUERY PLAN

```
Index Scan using bookings_pkey on bookings
  Index Cond: ((book_ref > '000900'::bpchar) AND (book_ref < '000939'::b
(2 rows)
```

Нет дополнительного узла
(дополнительных усилий) на сортировку

Тот же самый индекс может использоваться и для получения строк в обратном порядке:

```
=> EXPLAIN (analyze, buffers, costs off, timing off, summary off)
SELECT * FROM bookings
WHERE book_ref > '000900' AND book_ref < '000939'
ORDER BY book_ref DESC;
```

QUERY PLAN

```
-----
Index Scan Backward using bookings_pkey on bookings (actual rows=5 ]
  Index Cond: ((book_ref > '000900'::bpchar) AND (book_ref < '000939'::bpchar))
  Buffers: shared hit=4 read=1
Planning:
  Buffers: shared hit=12
(5 rows)
```

В этом случае мы спускаемся от корня дерева к правому листовому узлу и проходим по списку листовых страниц в обратную сторону.

Тот же самый индекс может использоваться и для получения строк в обратном порядке:

```
=> EXPLAIN (analyze, buffers, costs off, timing off, summary off)
SELECT * FROM bookings
WHERE book_ref > '000900' AND book_ref < '000939'
ORDER BY book_ref DESC;
```

QUERY PLAN

```
-----
Index Scan Backward using bookings_pkey on bookings (actual rows=5 ]
  Index Cond: ((book_ref > '000900'::bpchar) AND (book_ref < '000939'::bpchar))
  Buffers: shared hit=4 read=1
Planning:
  Buffers: shared hit=12
(5 rows)
```

количество страниц (Buffers),
которое потребовалось
прочитать.

```
=> EXPLAIN (analyze, buffers, costs off)
SELECT * FROM bookings
WHERE book_ref IN ('000906', '000909', '000917', '000930', '000938')
ORDER BY book_ref DESC;
```

QUERY PLAN

```
-----
-----
Index Scan Backward using bookings_pkey on bookings (actual time=0.038..0.040
loops=1)
  Index Cond: (book_ref = ANY ('{000906,000909,000917,000930,000938}'::bpchar))
  Buffers: shared hit=24
Planning Time: 0.084 ms
Execution Time: 0.075 ms
(5 rows)
```

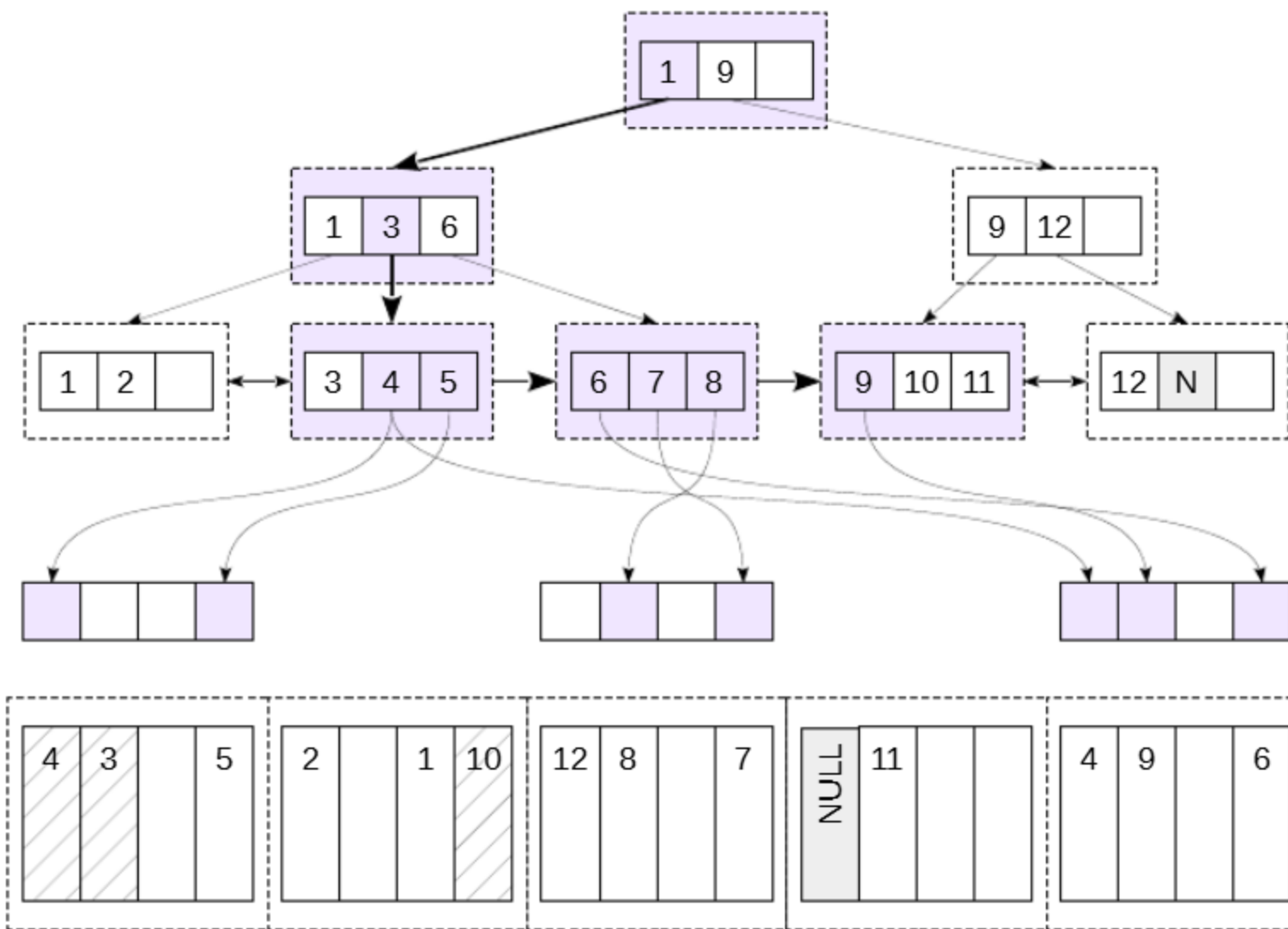
- Чтобы не тратить ресурсы на повторный просмотр табличных страниц, применяется еще один способ доступа — **сканирование по битовой карте**. Он похож на обычный индексный доступ, но происходит в два этапа.
- Сначала сканируется индекс (Bitmap Index Scan) и в локальной памяти процесса строится **битовая карта**. Битовая карта состоит из фрагментов. Фрагменты соответствуют табличным страницам, *а каждый бит фрагмента соответствует версии строки в этой странице*. При построении битовой карты в ней отмечаются те версии строк, которые удовлетворяют условию и должны быть прочитаны.

Когда индекс просканирован и битовая карта готова, начинается сканирование таблицы (Bitmap Heap Scan).

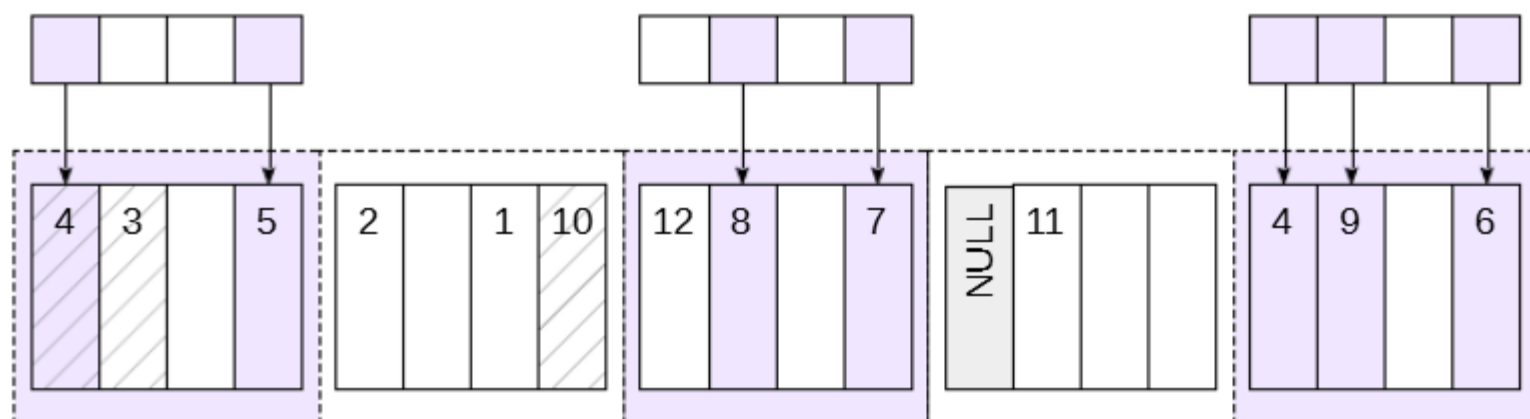
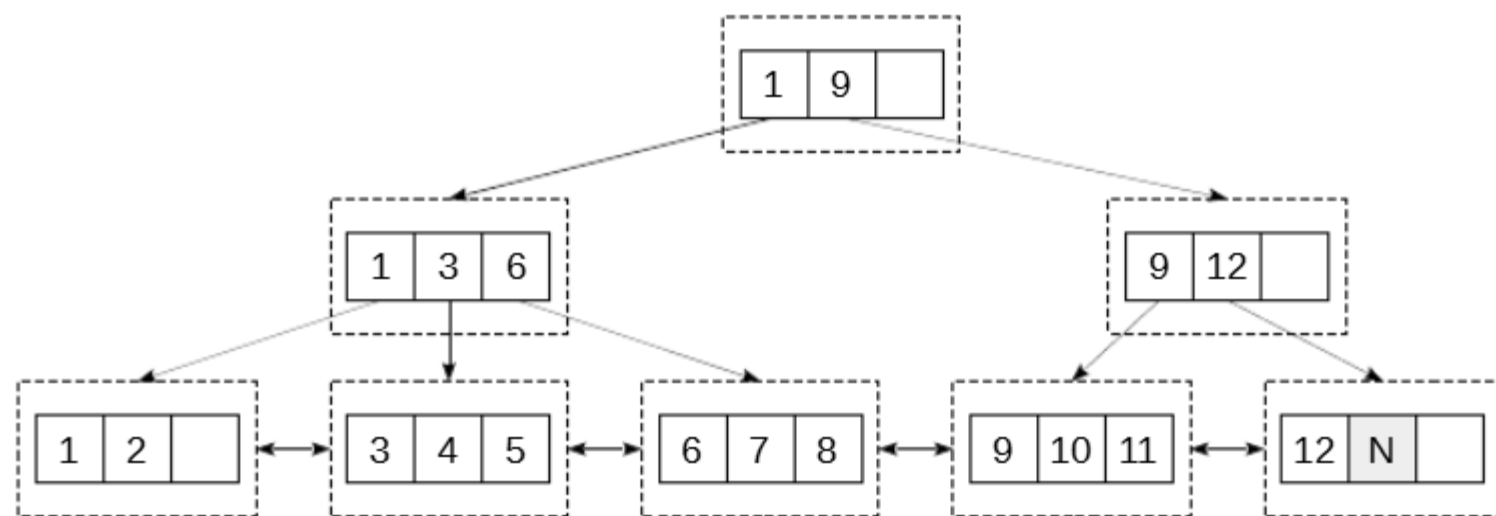
При этом:

- используется собственный механизм предвыборки, при котором асинхронно читаются *effective_io_concurrency* страниц (по умолчанию значение равно единице);
- на одной странице может проверяться несколько версий строк, но каждая страница просматривается ровно один раз.

Bitmap Index Scan



Bitmap Heap Scan



Будем рассматривать таблицу бронирований bookings. Создадим на ней два дополнительных индекса:

=> `CREATE INDEX ON bookings(book_date);`

CREATE INDEX

=> `CREATE INDEX ON bookings(total_amount);`

CREATE INDEX

=> `EXPLAIN`

`SELECT * FROM bookings WHERE total_amount < 5000;`

QUERY PLAN

```
-----  
--  
Bitmap Heap Scan on bookings (cost=80.62..8941.11 row  
  Recheck Cond: (total_amount < '5000'::numeric)  
    -> Bitmap Index Scan on bookings_total_amount_idx  
width=0)  
      Index Cond: (total_amount < '5000'::numeric)  
(4 rows)
```

```
=> EXPLAIN (costs off)
SELECT * FROM bookings
WHERE total_amount < 5000 OR total_amount > 500000;
```

QUERY PLAN

Bitmap Heap Scan on bookings

Recheck Cond: ((total_amount < '5000'::numeric) OR (total_amount > '500000'::numeric))

-> BitmapOr

-> Bitmap Index Scan on bookings_total_amount_idx

Index Cond: (total_amount < '5000'::numeric)

-> Bitmap Index Scan on bookings_total_amount_idx

Index Cond: (total_amount > '500000'::numeric)

(7 rows)

Таким же образом могут быть использованы и разные индексы:

```
=> EXPLAIN (costs off)
SELECT * FROM bookings
WHERE total_amount < 5000
      OR book_date = bookings.now() - INTERVAL '1 day';
```

QUERY

```
-----
-----
Bitmap Heap Scan on bookings
  Recheck Cond: ((total_amount < '5000'::numeric) OR (book_date = ('2017-08-15 18:00:00+03'::timestamp with time zone - '1 day'::interval)))
   -> BitmapOr
        -> Bitmap Index Scan on bookings_total_amount_idx
              Index Cond: (total_amount < '5000'::numeric)
        -> Bitmap Index Scan on bookings_book_date_idx
              Index Cond: (book_date = ('2017-08-15 18:00:00+03'::timestamp with time zone - '1 day'::interval))
(7 rows)
```

Неточные фрагменты



Битовая карта без потери точности

пока размер карты не превышает *work_mem*,
информация хранится с точностью до версии строки

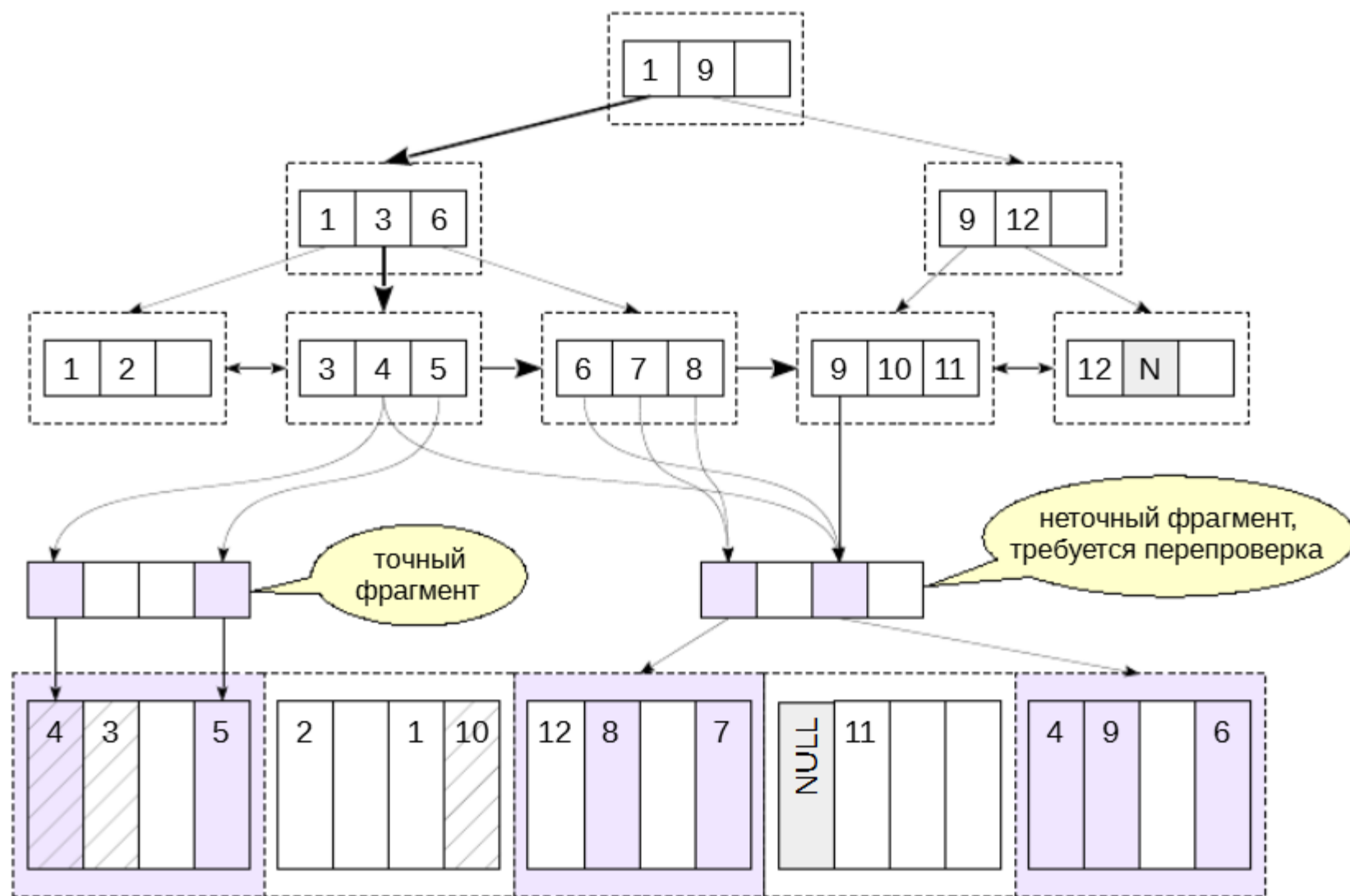
по умолчанию 4MB



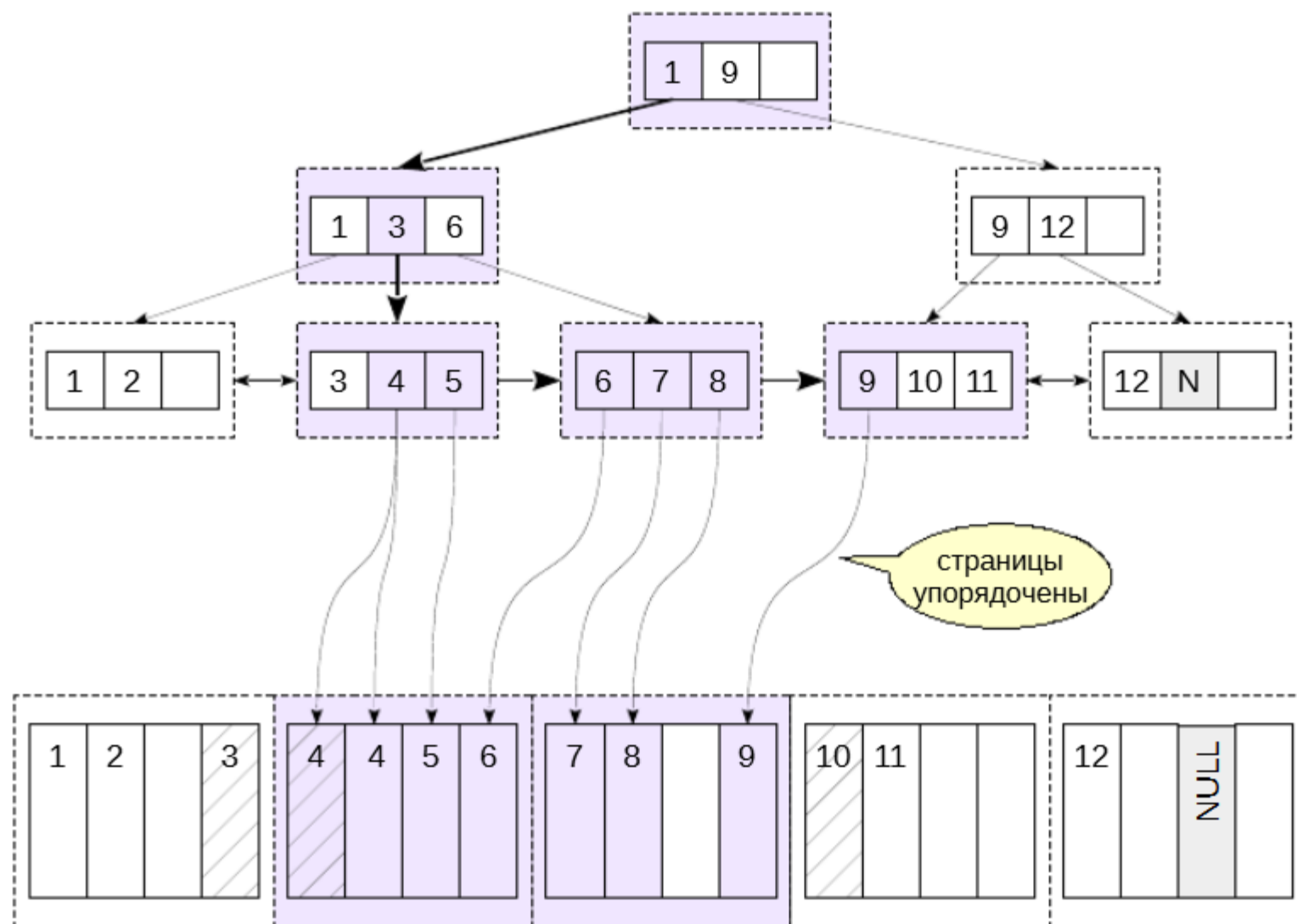
Битовая карта с потерей точности

если память закончилась, происходит огрубление
части уже построенной карты до отдельных страниц
требуется примерно 1 МБ памяти на 64 ГБ данных;
ограничение *work_mem* может быть превышено

Неточные фрагменты

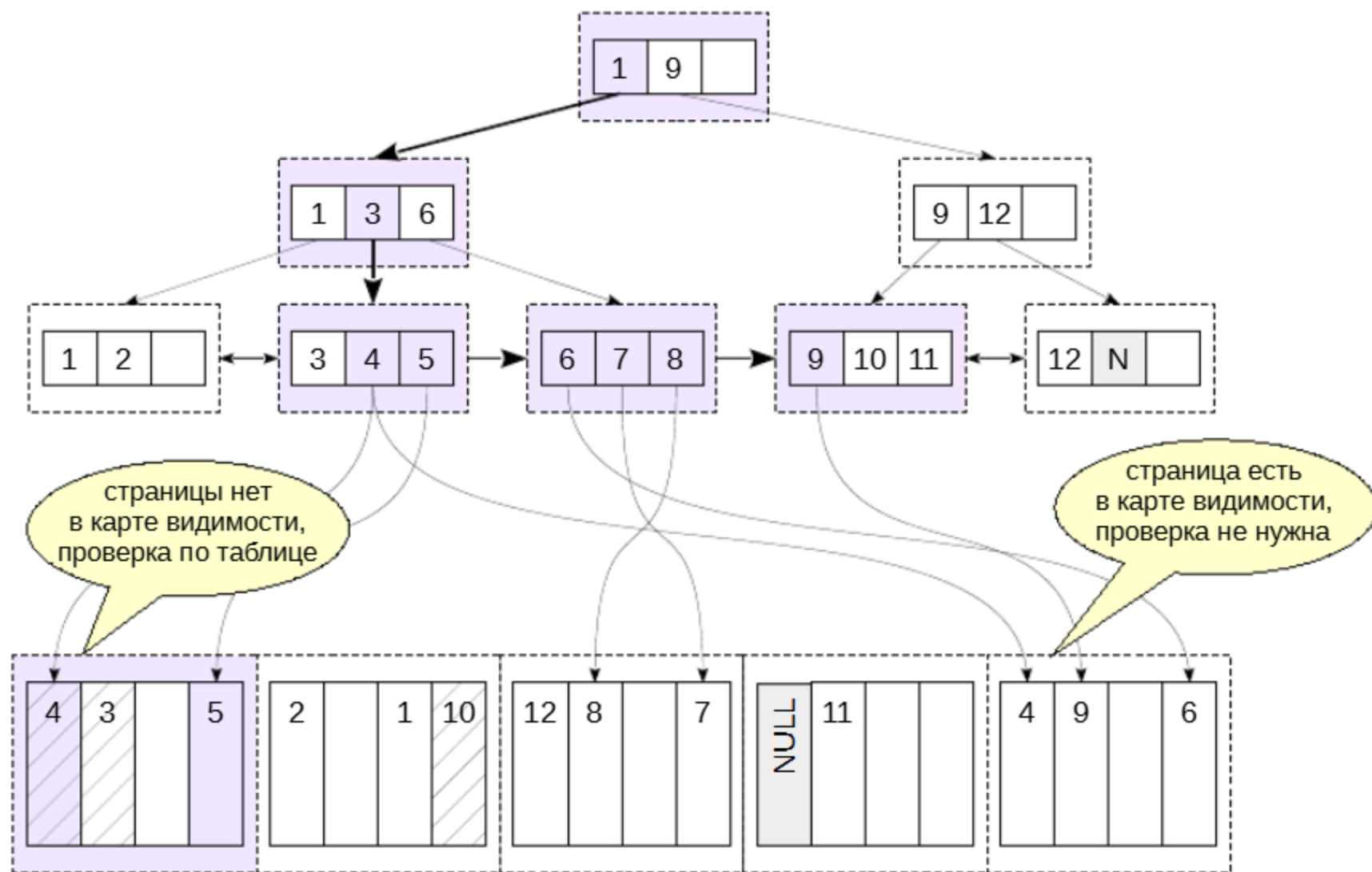


Index Scan / Bitmap Scan



- Если данные в таблице физически упорядочены, обычное индексное сканирование не будет читать табличную страницу повторно.
- В таком (нечастом на практике) случае метод сканирования по битовой карте теряет смысл, проигрывая обычному индексному сканированию.
- Разумеется, планировщик это тоже учитывает

Index-Only Scan



```
=> EXPLAIN (costs off)
SELECT total_amount FROM bookings
WHERE total_amount > 200000;
```

QUERY PLAN

```
-----
Index Only Scan using bookings_total_amount_idx on bookings
    Index Cond: (total_amount > '200000'::numeric)
(2 rows)
```

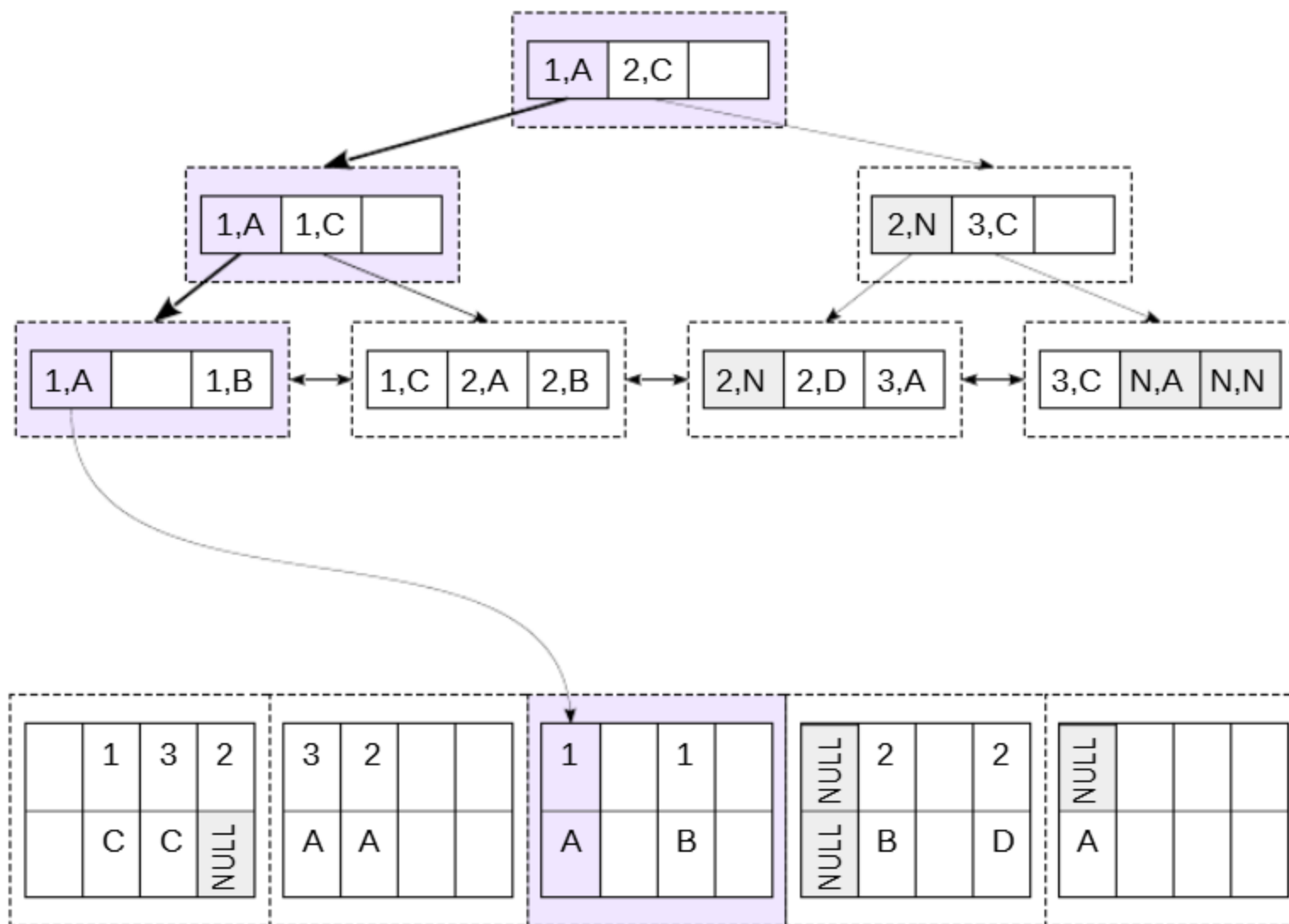
```
=> EXPLAIN (analyze, costs off, timing off, summary off)
SELECT total_amount FROM bookings
WHERE total_amount > 200000;
```

QUERY PLAN

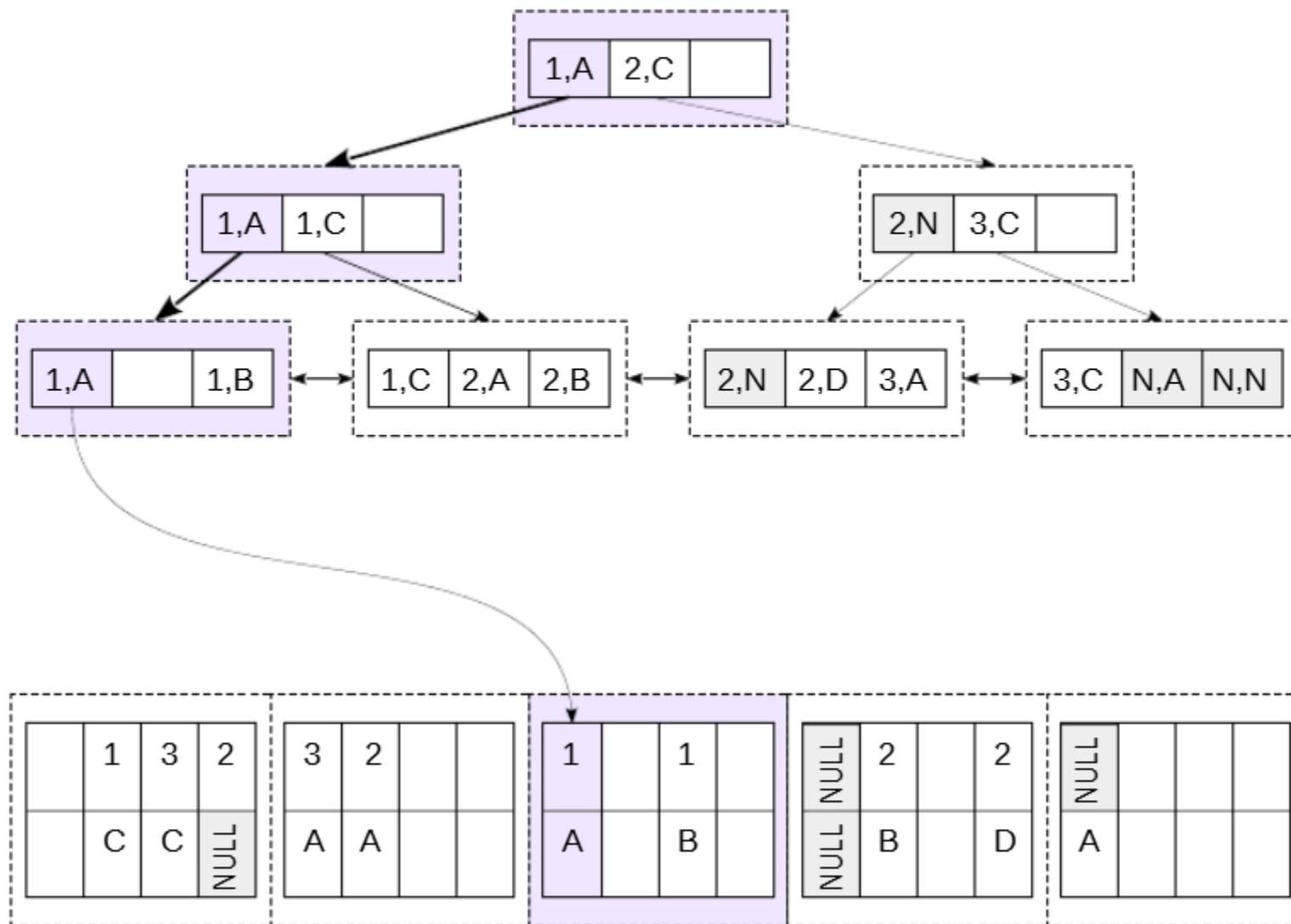
```
-----
Index Only Scan using bookings_total_amount_idx on bookings (actual
  Index Cond: (total_amount > '200000'::numeric)
  Heap Fetches: 0
(3 rows)
```

Строка Heap Fetches показывает, сколько версий строк было проверено с помощью таблицы. В данном случае карта видимости содержит актуальную информацию, обращаться к таблице не потребовалось.

Многоколоночный индекс

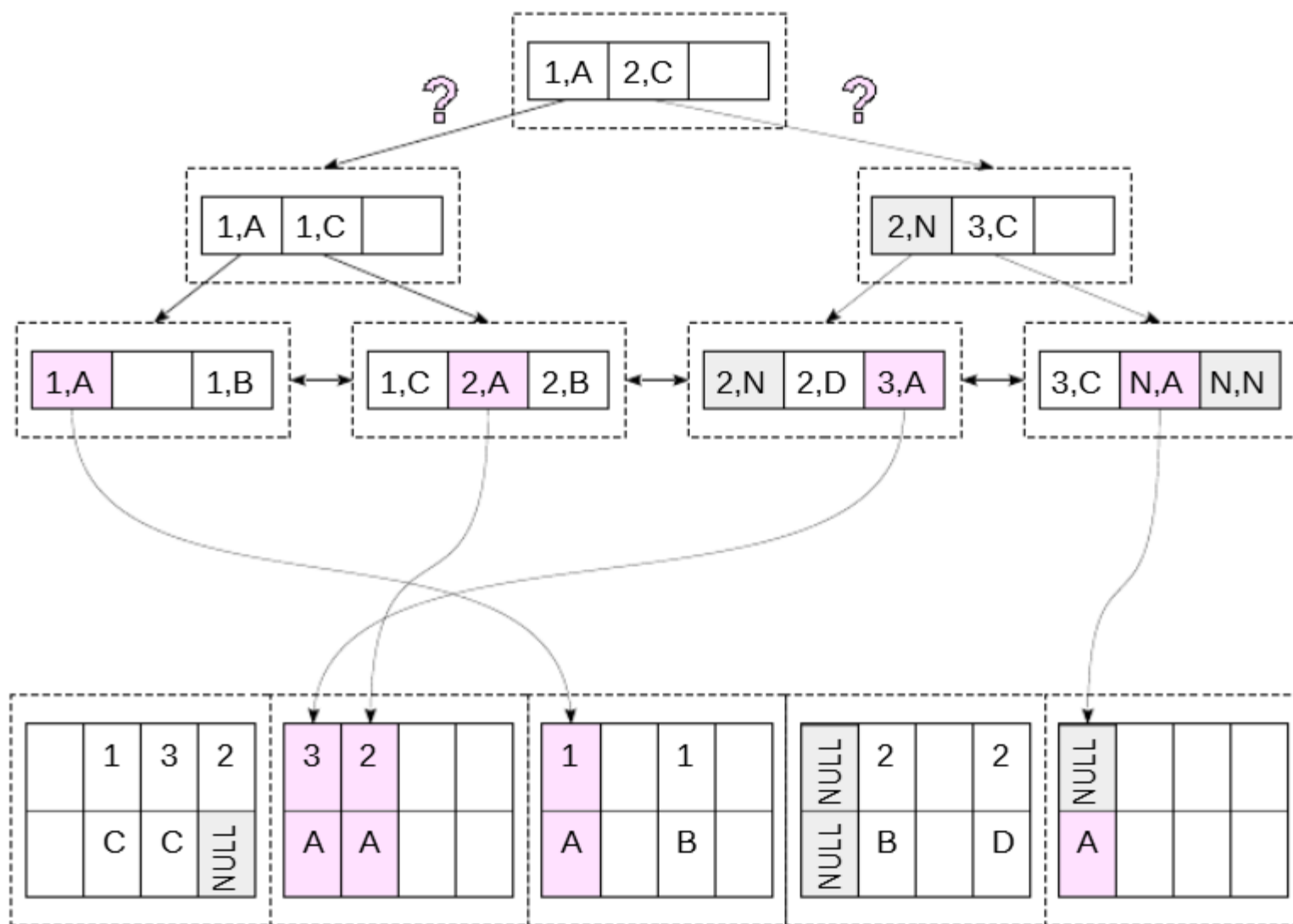


Многоколоночный индекс



Такой индекс ускоряет поиск, если в запросе есть условие на один или несколько первых ключей, поскольку ключи отсортированы сначала по первому столбцу, затем по второму и т.д.

Многоколоночный индекс



Индекс с дополнительными неключевыми столбцами

```
CREATE INDEX ... INCLUDE (...)
```

Неключевые столбцы

- не используются при поиске по индексу

- не учитываются ограничением уникальности

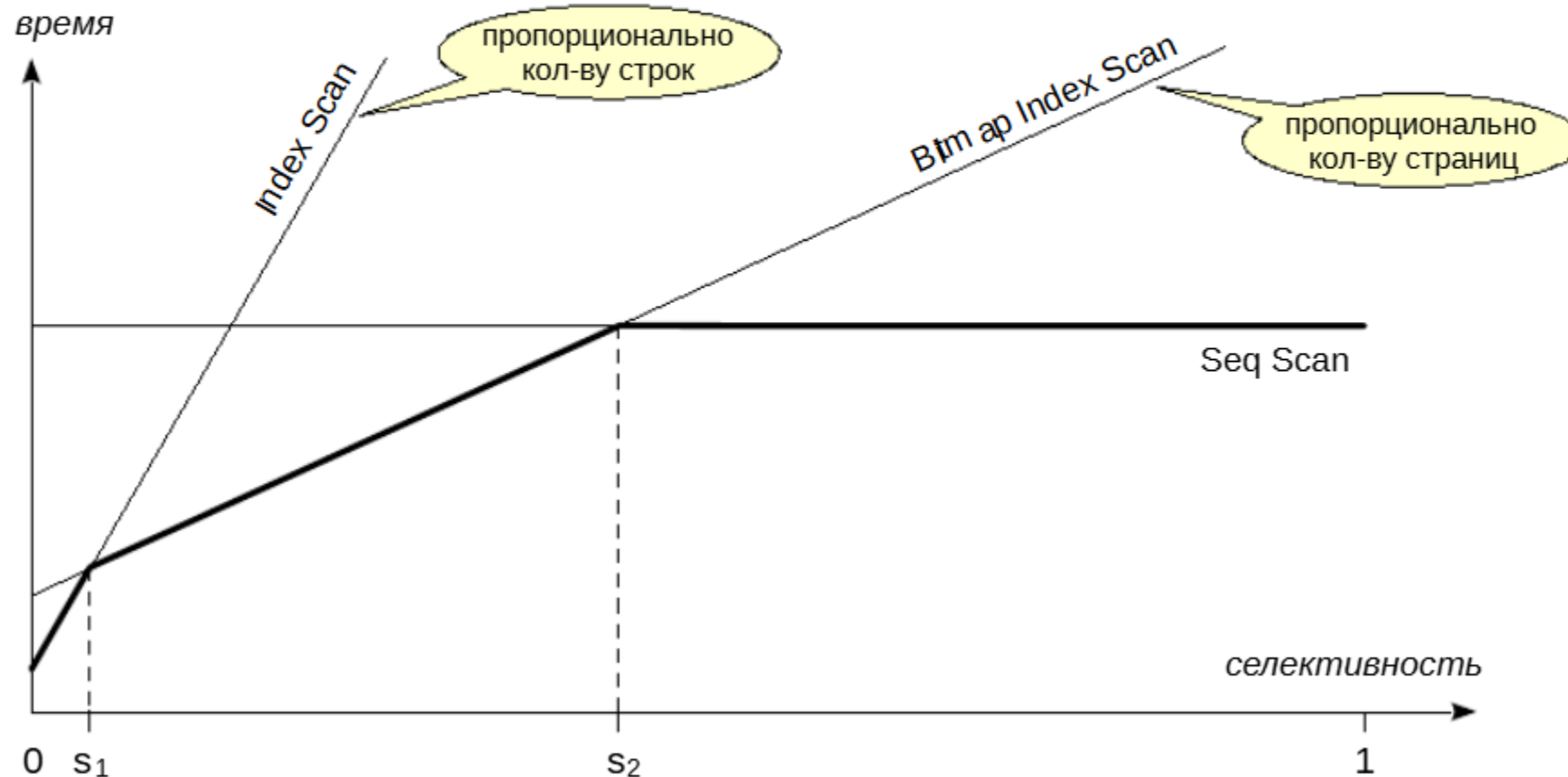
- значения хранятся в индексной записи

- и возвращаются без обращения к таблице

=> **CREATE UNIQUE INDEX ON tickets (ticket_no) INCLUDE (book_ref);**

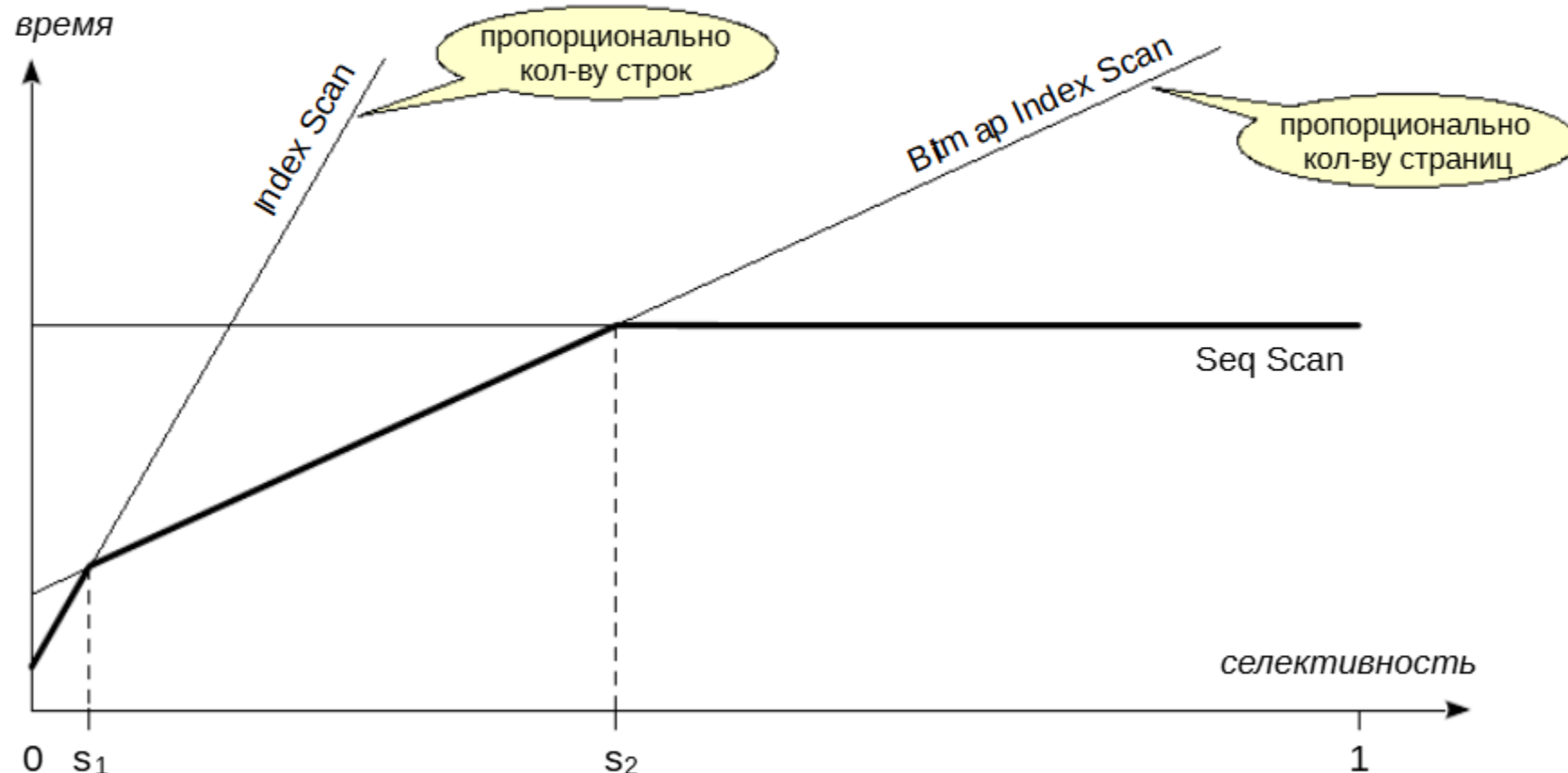
В include-индекс можно включать столбцы с типами данных, которые не поддерживаются B-деревом (геометрические типы, xml и др.).

Сравнение эффективности



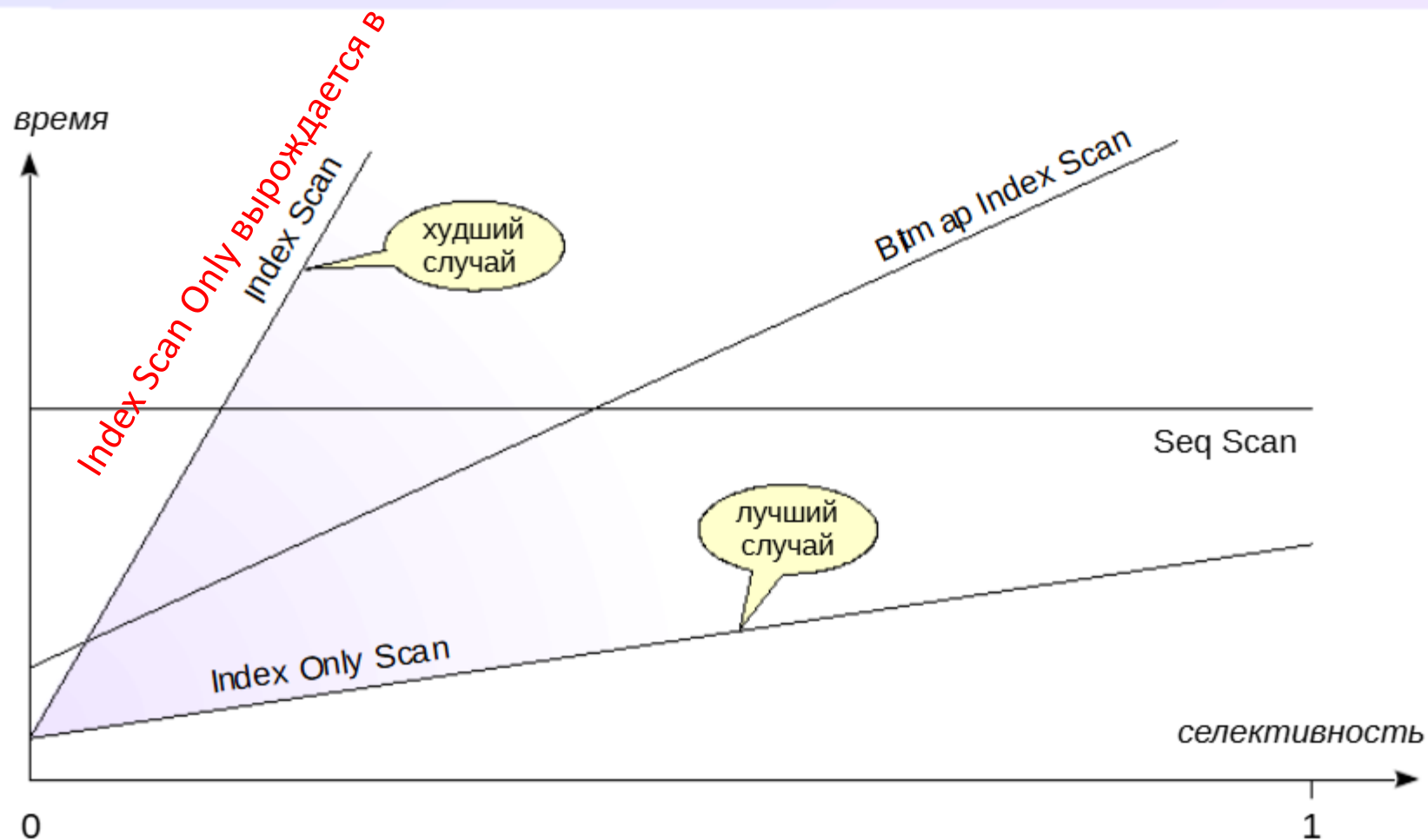
Селективность – процент «выборности» строк запросом –
число выбранных запросом строк / число всех строк в индексируемой таблице

Сравнение эффективности



↑
↑
сильно зависят от конкретной таблицы и конкретного индекса

Сравнение эффективности



Оптимизатор располагает несколькими методами доступа

- последовательное сканирование

- сканирование индекса

- сканирование только индекса

- сканирование по битовой карте

Модель стоимости учитывает множество параметров



БАЗЫ ДАННЫХ

Лекция 3.

Виды индексов в PG

Типы индексов в PG

- B-tree
- Хеш-индекс
- GiST
- SP-GiST
- GIN
- BRIN

B-Tree индекс

- Наиболее часто используемый индекс
- Для равенства и диапазонных запросов (=, <, >, BETWEEN)
- Покрывает 90% потребностей в запросах

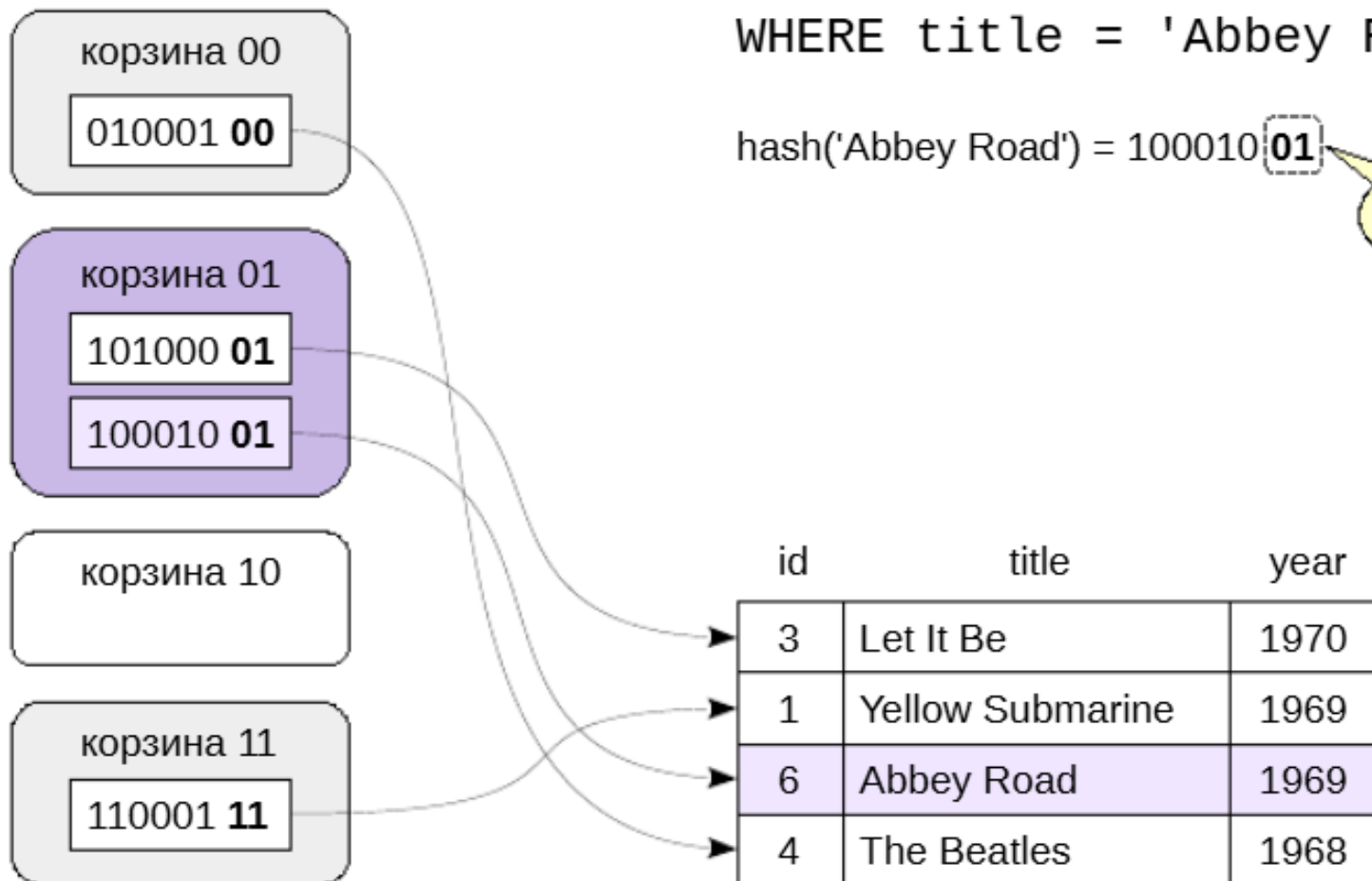
CREATE INDEX idx_name ON table(column);

Hash индекс

```
CREATE INDEX ON t USING hash(title);  
SELECT * FROM t  
WHERE title = 'Abbey Road';
```

hash('Abbey Road') = 100010**01**

номер
корзины



Hash индекс

- Хранит только хеш-коды, но не исходные значения
- Размер не зависит от ключа индексирования
- Невозможно сканирование только индекса
- Индекс увеличивается динамически (скачкообразный рост)

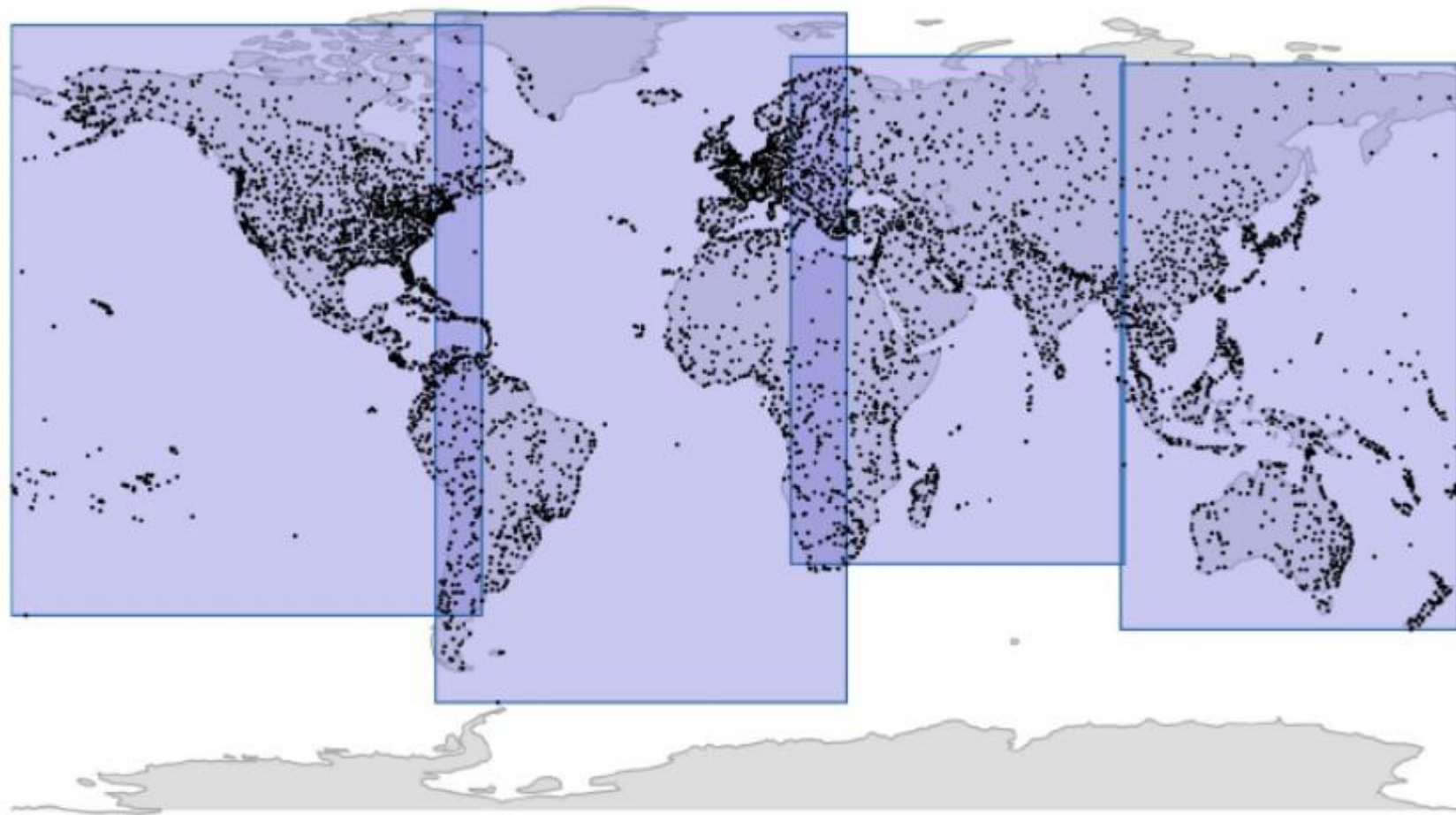
Hash индекс

В отличие от B-дерева, хеш-индексы имеют много ограничений:

- единственная поддерживаемая операция — поиск по условию **равенства**, поскольку хеш-функция не сохраняет порядок следования значений;
- не поддерживается ограничение **уникальности**;
- **нельзя** создать **многоколоночный** индекс и добавить к индексу дополнительные include-столбцы.

Хеш-индексы не получили широкого распространения. Однако за счет меньшего размера и фиксированного времени поиска хеш-индекс может в ряде случаев работать **быстрее**, чем индекс на основе B-дерева

GiST индекс



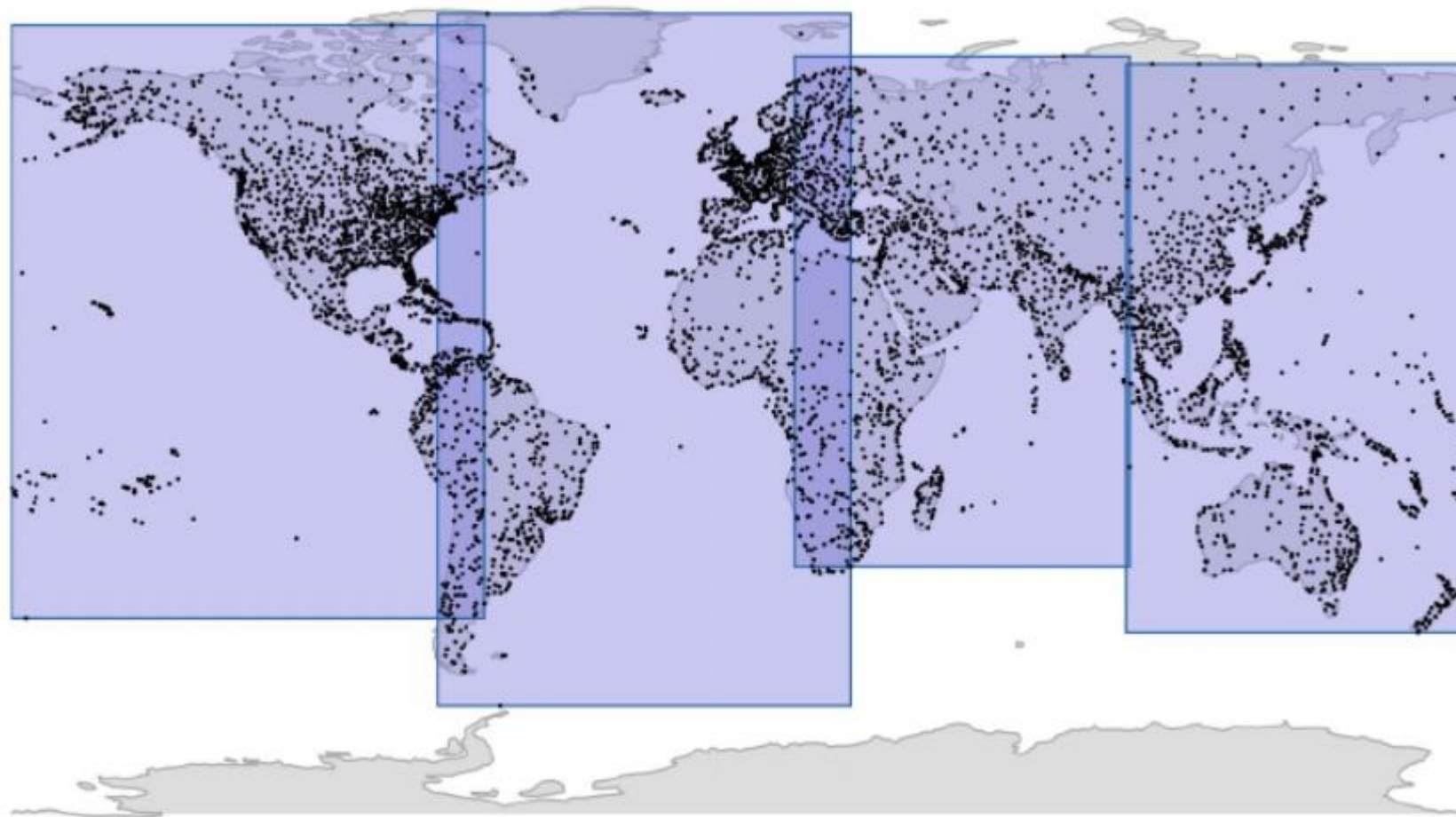
GiST индекс

- GiST расшифровывается как **generalized search tree** — обобщенное дерево поиска.
- Поддерживает геометрические объекты, полнотекстовый поиск

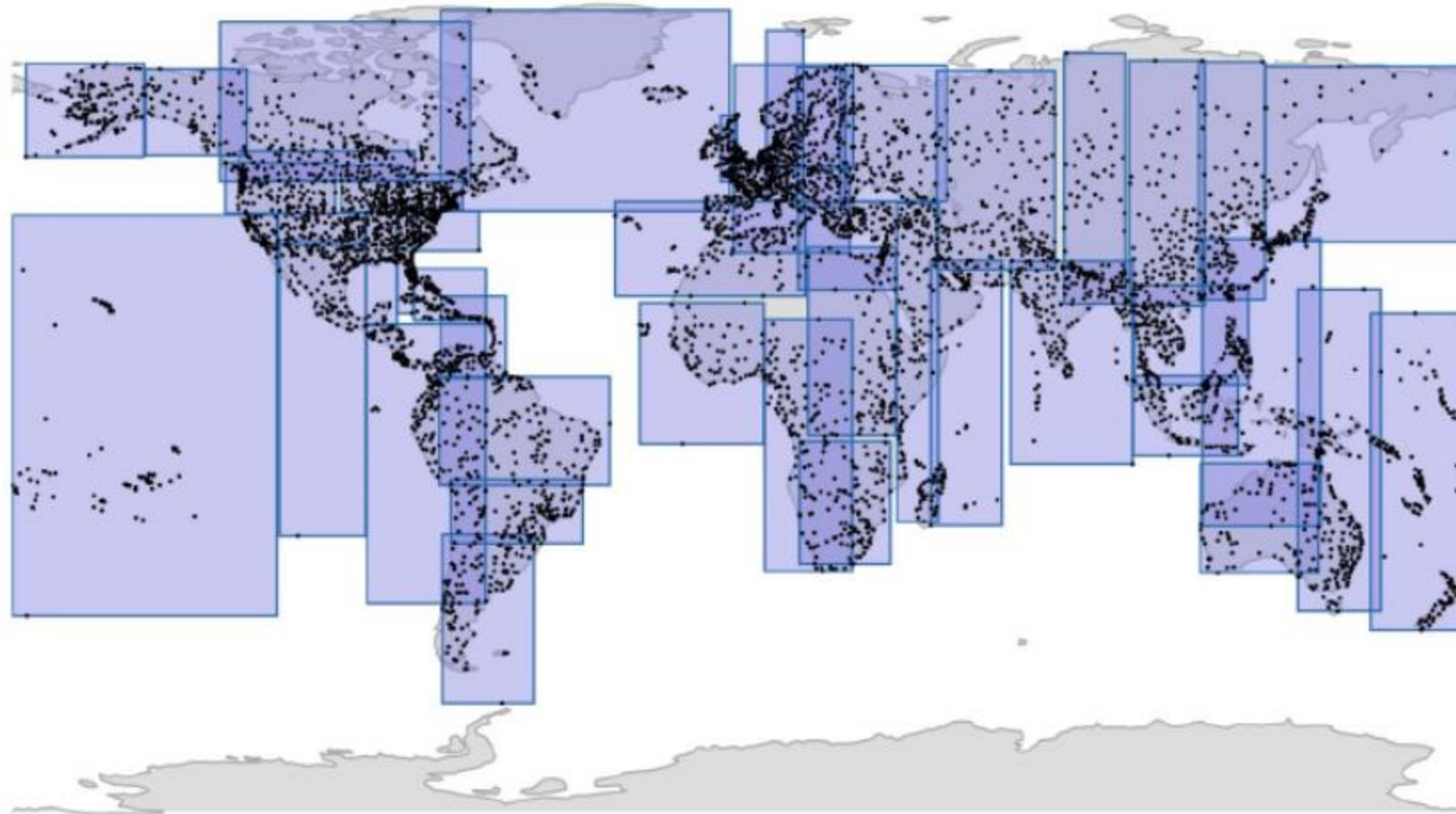
CREATE INDEX idx_gist_name ON table USING gist(column);

Идею работы GiST-индексов рассмотрим на примере точек на плоскости. Плоскость разбивается на несколько прямоугольников, которые в сумме покрывают все индексируемые точки. Эти прямоугольники составляют верхний уровень дерева. Как видно на рисунке, прямоугольники могут пересекаться (хотя это и уменьшает эффективность поиска). Такой алгоритм индексирования называется R-деревом.

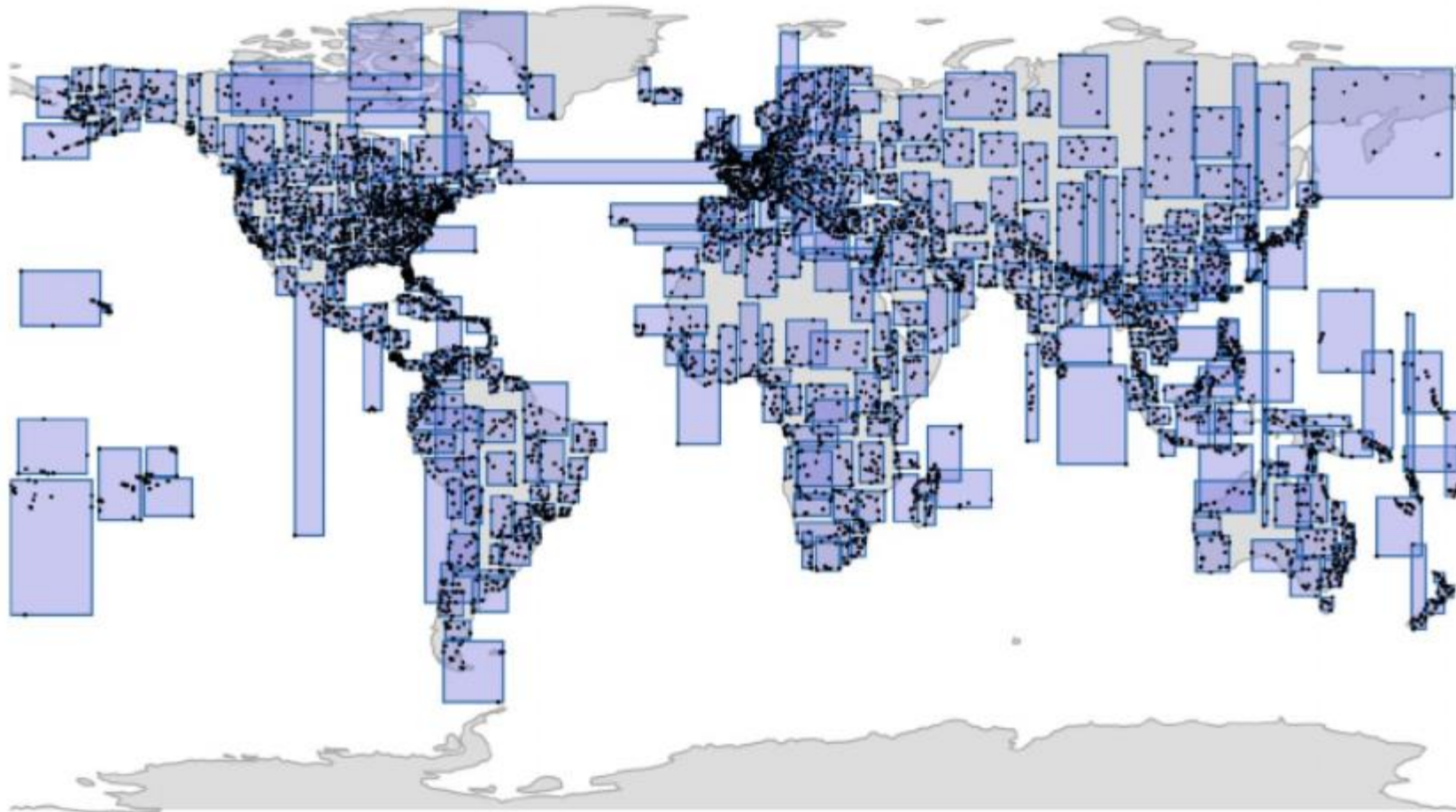
GiST индекс



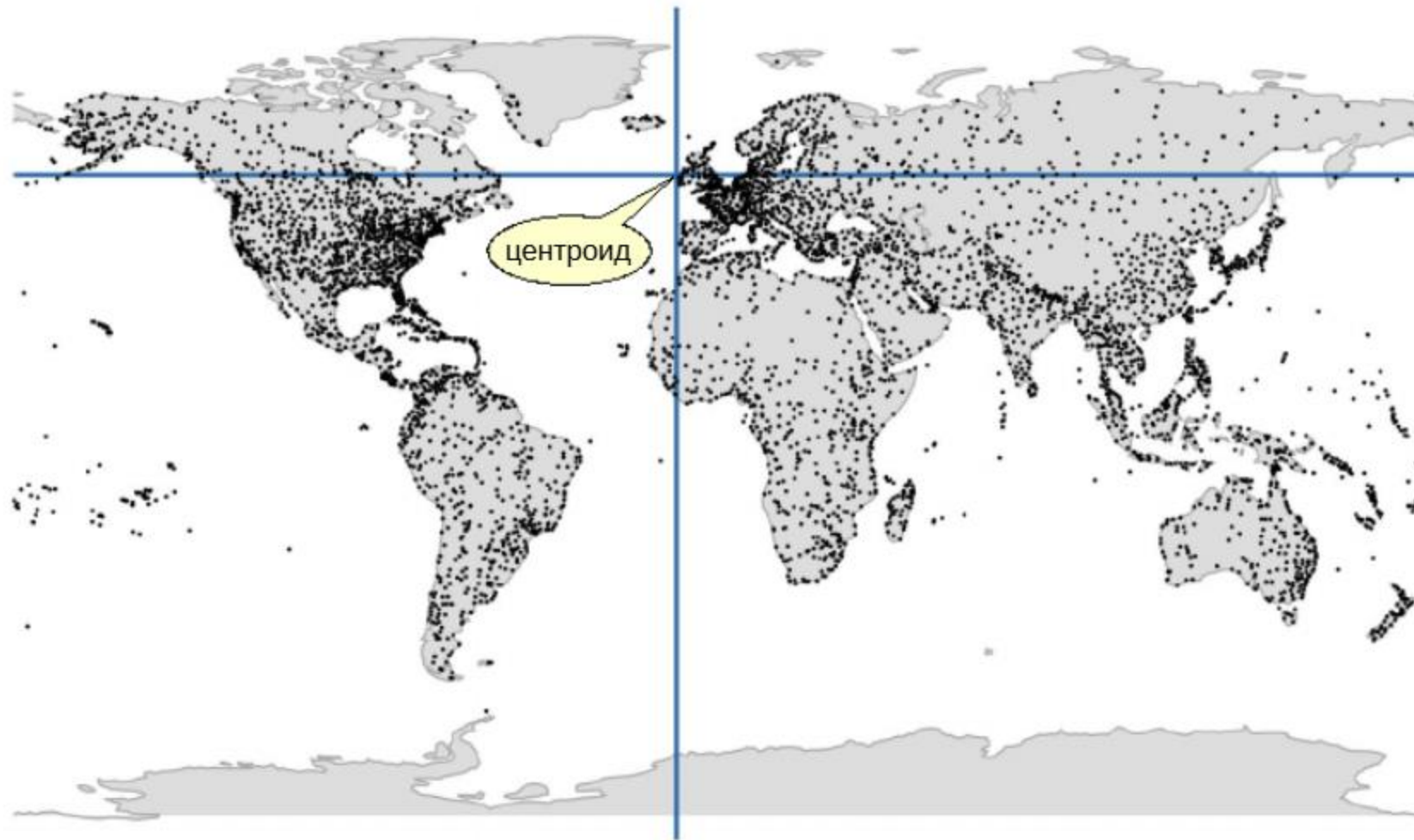
GiST индекс



GiST индекс



SP-GiST индекс



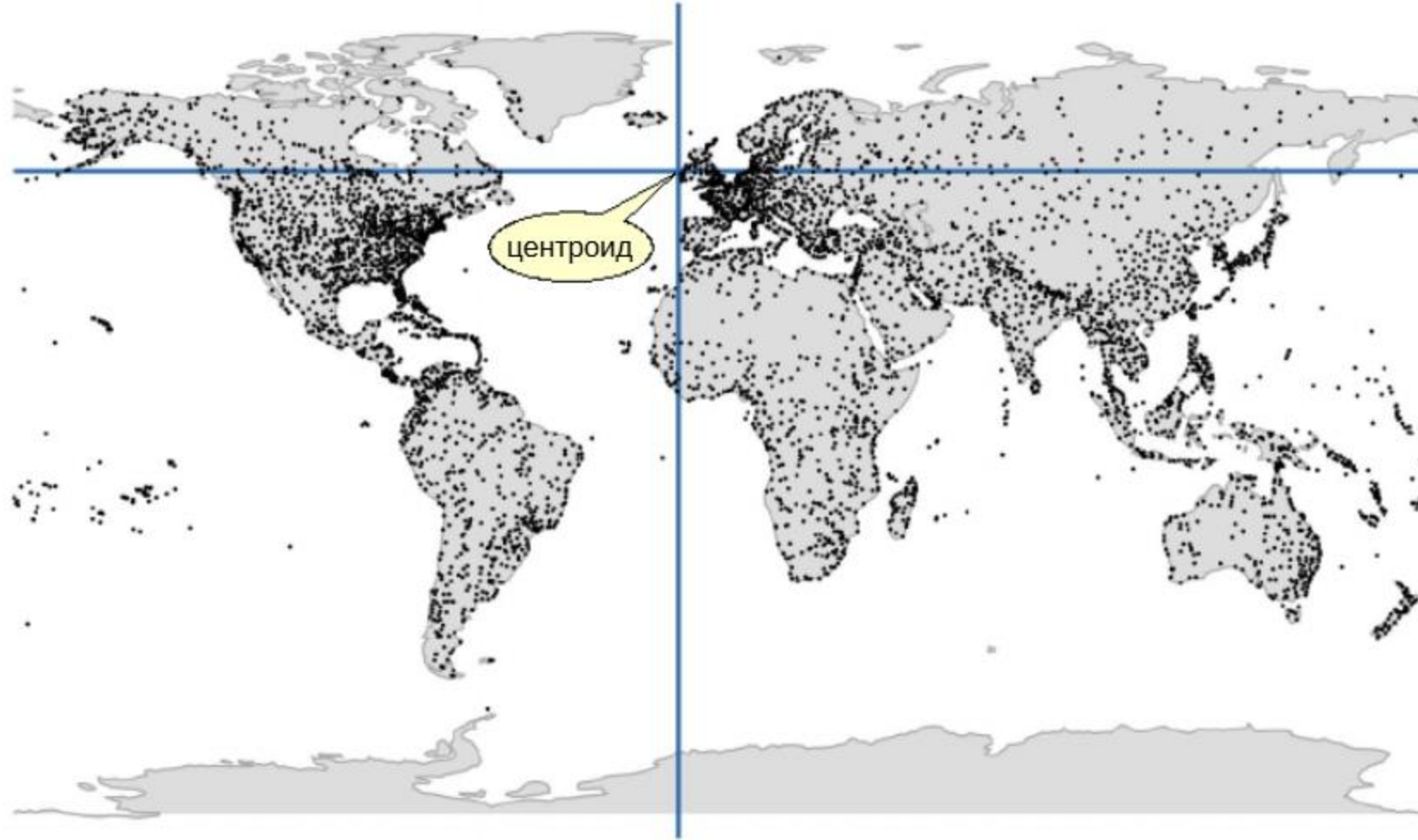
SP-GiST индекс

- SP-GiST расшифровывается как **space partitioning GiST**. Это тоже обобщенное дерево поиска, но оно строится путем разбиения пространства поиска на непересекающиеся области.
- Подходит для деревьев, IP-адресов, телефонных номеров

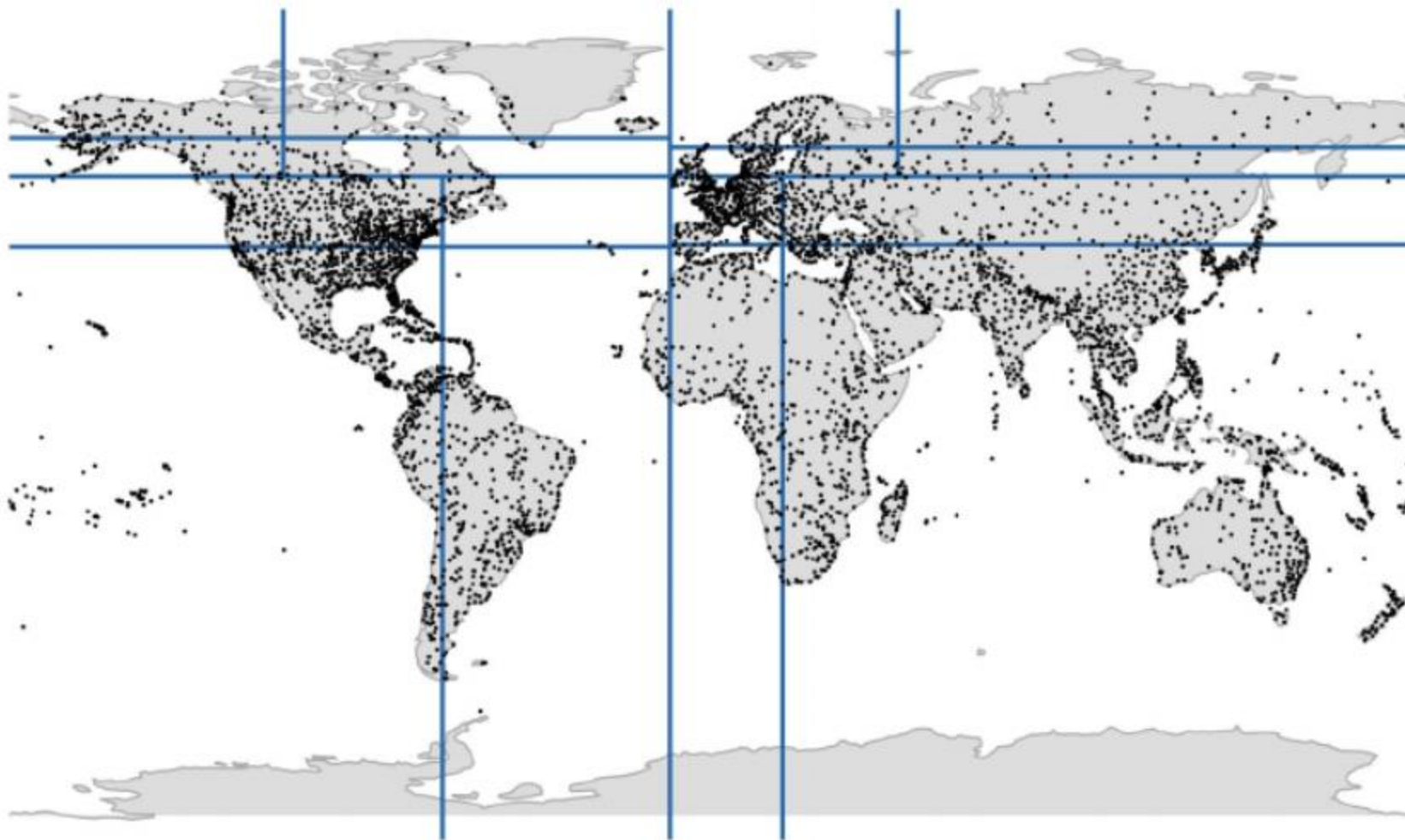
CREATE INDEX idx_spgist_name ON table USING spgist(column);

Рассмотрим пример индекса SP-GiST для точек на плоскости. Один из вариантов — дерево квадрантов. Корневой узел разбивает плоскость на четыре части (квадранта) по отношению к выбранной точке-центроиду.

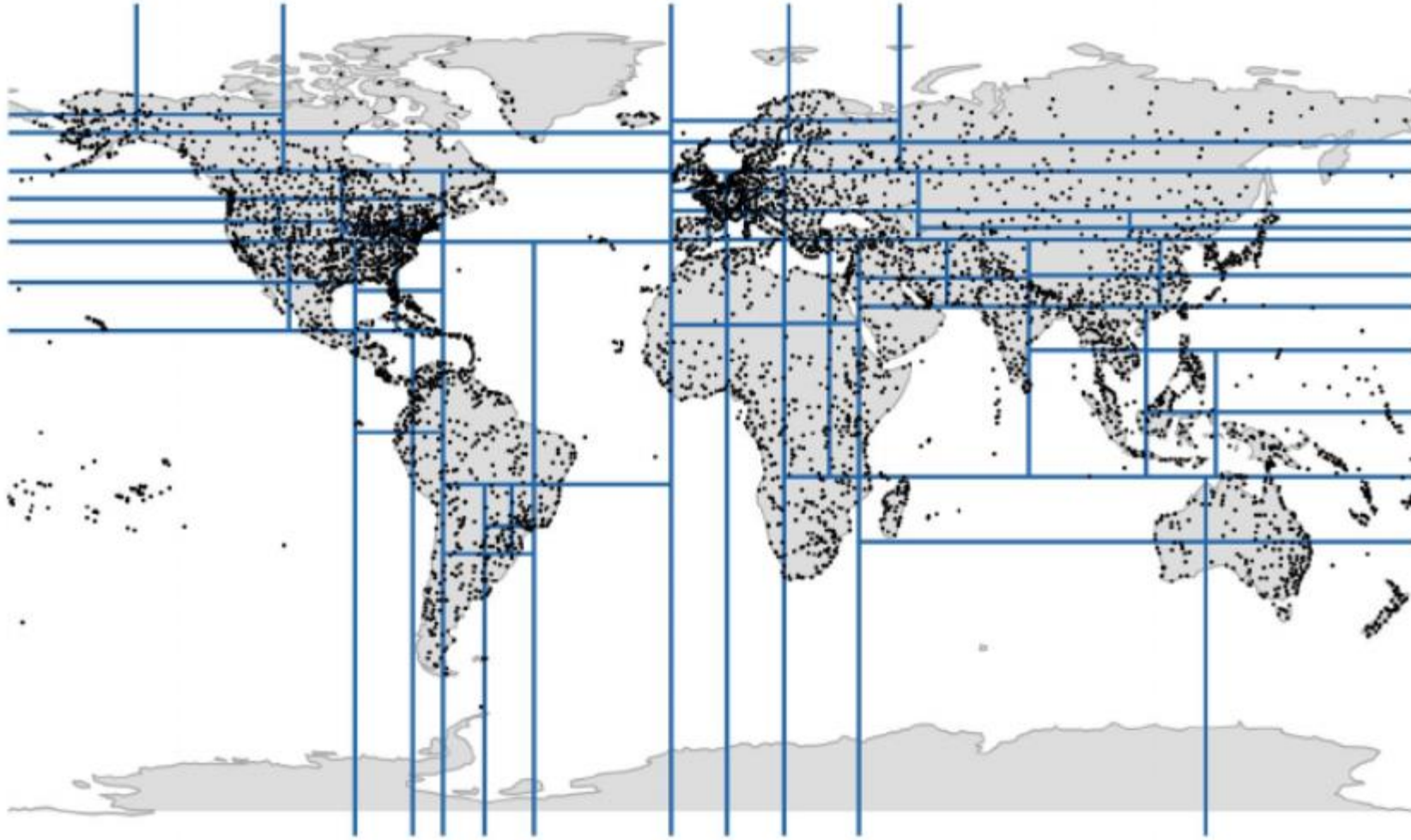
SP-GiST индекс



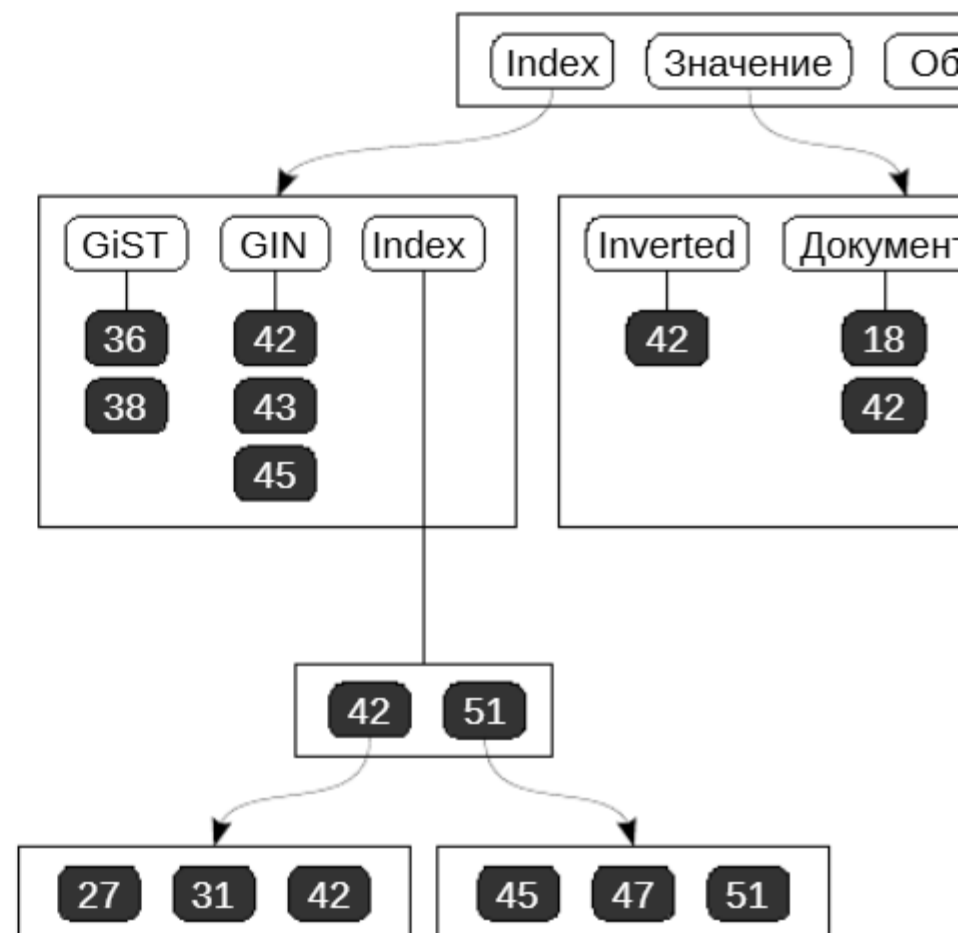
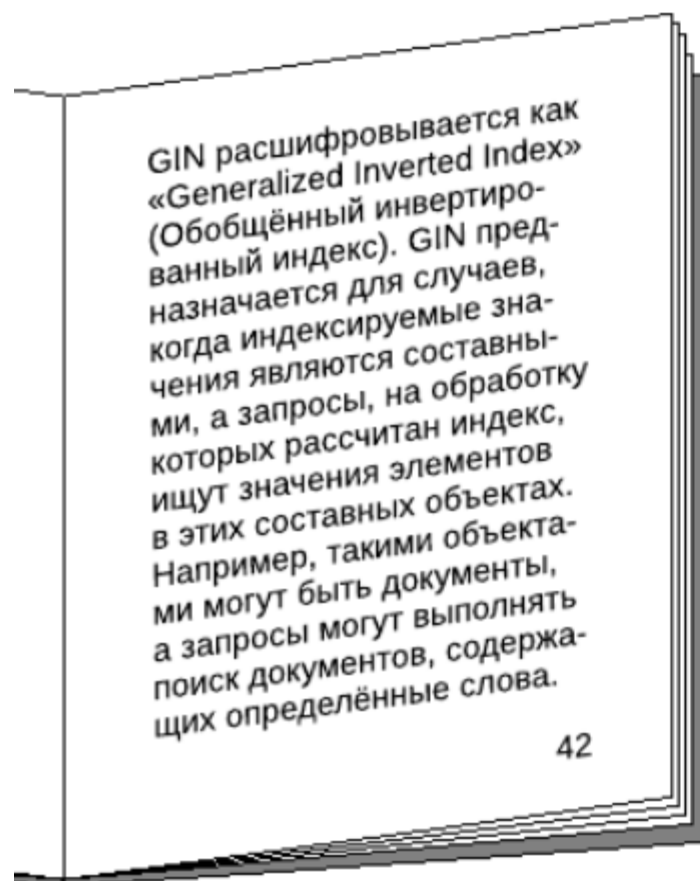
SP-GiST индекс



SP-GiST индекс



GIN индекс



GIN индекс

- GIN — **generalized inverted index**, обобщенный инвертированный индекс.
- Эффективен для массивов и **полнотекстового поиска**, быстро ищет наличие элементов
- Проверка соответствия документа поисковому запросу
- Проверка вхождения элемента в массив
- Поиск документов JSON по ключам или значениям

CREATE INDEX idx_gin_name ON table USING gin(column);

GIN индекс

Индекс GIN в первую очередь используется для **индексации документов с целью ускорения полнотекстового поиска.**

По сути, это обычное B-дерево, но в него помещены не сами документы, а составляющие их слова.

GIN-индекс оптимизирован с учетом того, что каждое слово может встречаться во многих документах. Если «список страниц» очень велик, он может храниться не в самой индексной странице, а в отдельном B-дереве.

Идею этого индексного метода проще всего понять на примере предметного указателя в обычной книге. На страницах книги встречаются термины, а в предметном указателе перечислены в алфавитном порядке все термины с указанием номеров страниц, на которых они присутствуют.

BRIN индекс

```
SELECT * FROM t WHERE temperature BETWEEN 20 AND 30;
```

группа
последовательно
расположенных
страниц

зона	страницы	сводная информация
1	1 .. 128	{ -15 .. -3 }
2	129 .. 256	{ -7 .. -10 }
3	257 .. 384	{ 6 .. 19 }
4	385 .. 512	{ 15 .. 25 }
5	513 .. 640	{ 21 .. 32 }
6	641 .. 768	{ 13 .. 19 }
7	769 .. 896	{ 3 .. 15 }
8	897 .. 1024	{ -5 .. 6 }
9	1025 .. 1152	{ -18 .. -4 }

BRIN индекс

- Сжимает данные по диапазонам

CREATE INDEX idx_brin_name ON table USING brin(column);

Список зон со сводной информацией:

- зона охватывает группу последовательно расположенных страниц
- сводная информация: минимум, максимум и т. п.
- Требуется корреляция с физическим расположением строк
- Не хранит ссылки на версии строк (только сканирование по битовой карте)
- Предназначен для очень больших таблиц (с упорядоченными данными)
- Небольшой размер
- Настраиваемое соотношение размера и точности

Тип	Лучше всего подходит	Операторы (примеры)	Аналогия
B-tree	Уникальные значения, диапазоны, сортировка	=, >, <, >=, <=, BETWEEN, LIKE 'abc%'	Алфавитный указатель в книге
Hash	Строгое равенство	=	Быстрый поиск по ключу в словаре
GiST	Геометрия, полнотекстовый поиск, диапазоны	&&, @>, <->, @@	Карта, разбитая на квадраты (с разной детализацией)
SP-GiST	Иерархические данные, IP-адреса	~, <<, >>=	Дерево телефонных кодов страны/города
GIN	Составные значения: массивы, JSON, полнотекстовый поиск	@>, @@, ?, `	Предметный указатель в конце книги (слово -> страницы)
BRIN	Огромные таблицы с физическим порядком	=, >, < (для упорядоченных данных)	Сжатие данных по диапазонам

Другие возможности

- Уникальные индексы (UNIQUE)
- Частичные индексы (WHERE ...)
- Многоколоновые индексы
- Индексы по выражениям

Частичные индексы

- Индексы создаются только для строк, удовлетворяющих условию WHERE
- Уменьшают размер и ускоряют запросы по подмножеству данных

```
CREATE INDEX idx_active_users ON users(last_login)  
WHERE active = true;
```

Многоколоновые индексы

- Индекс по нескольким колонкам
- Важно учитывать порядок колонок

```
CREATE INDEX idx_name_age ON people  
(last_name, first_name, age);
```

Индексы по выражениям

- Индексируются результаты вычисления выражения или функции
- Полезно для запросов с вычисляемыми условиями

```
CREATE INDEX idx_lower_email ON users(LOWER(email));
```

Операторы в общем виде

```
CREATE [ UNIQUE ] INDEX [ CONCURRENTLY ] [ IF NOT EXISTS ] имя_индекса  
ON [ ONLY ] имя_таблицы [ USING метод ]  
  
( { имя_столбца | ( выражение ) } [ COLLATE правило_сортировки ]  
[ класс_операторов [ ( параметр_класса_оп = значение [, ... ] ) ] ]  
[ ASC | DESC ] [ NULLS { FIRST | LAST } ] [, ... ] )  
  
[ INCLUDE ( имя_столбца [, ...] ) ]  
  
[ NULLS [ NOT ] DISTINCT ]  
  
[ WITH ( параметр_хранения [= значение] [, ... ] ) ]  
  
[ TABLESPACE табл_пространство ]  
  
[ WHERE предикат ]
```

```
DROP INDEX имя_индекса ON имя_таблицы;
```

Рекомендации по созданию индексов (общие!)

Факторы, определяющие «хороший» индекс:

- число столбцов в индексе не более 4-5
- в индекс включаются не часто обновляемые столбцы
- создание индексов по FK
- создание индексов по столбцам из выражения WHERE часто выполняемых запросов

Факторы, определяющие «плохой» индекс:

- по столбцам, входящих в индекс, не выполняются поиски строк
- слишком много столбцов в индексе
- слишком мало записей в индексируемой таблице
- слишком много индексов